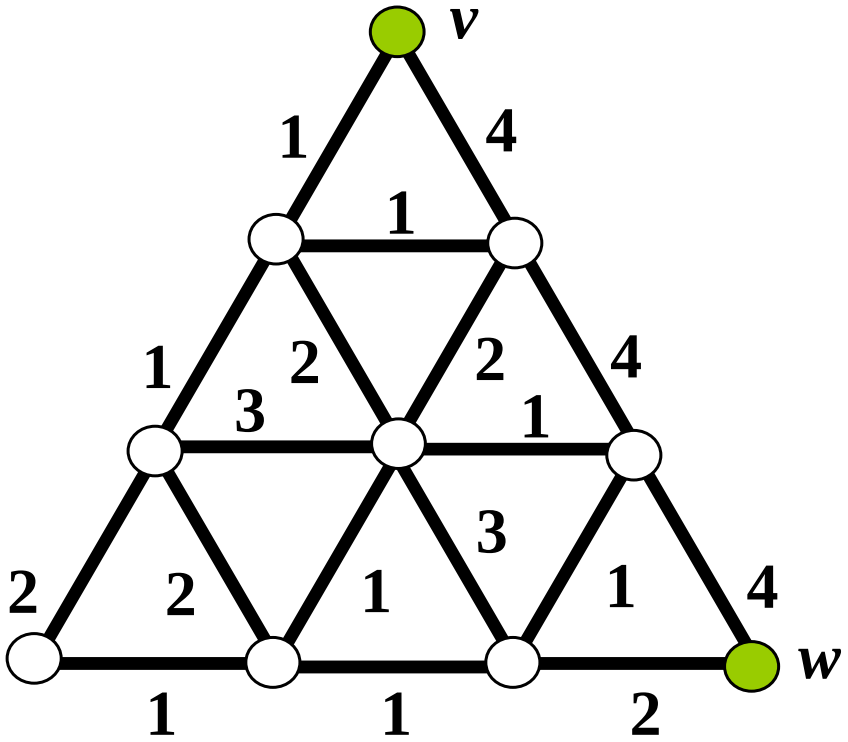
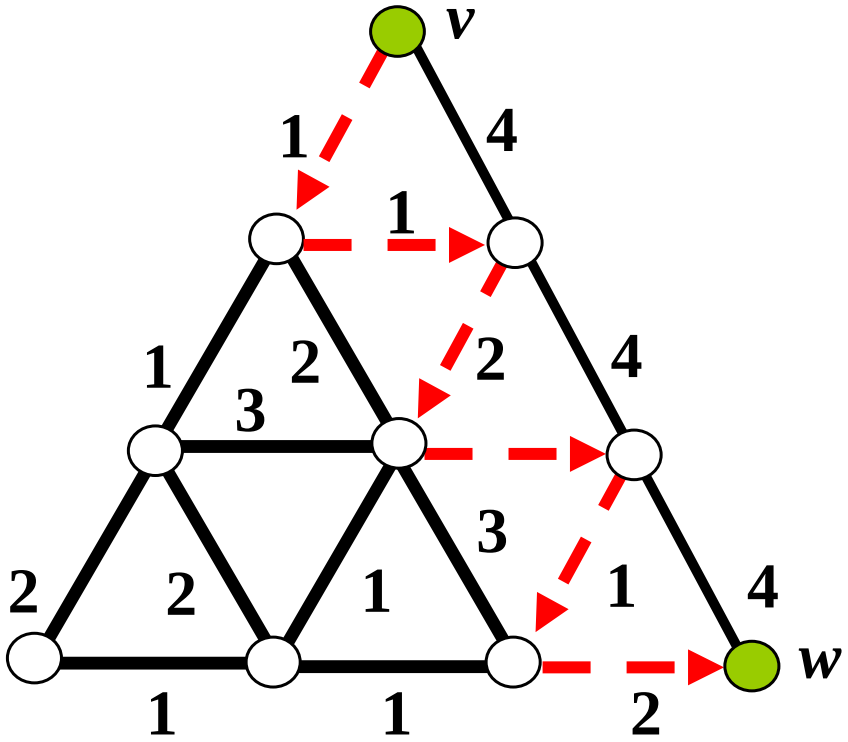


Caminho mais Curto

Caminho mais curto

O caminho mais curto **entre os vértices *v* e *w* de um grafo *G* ponderado é aquele que acumula o menor valor possível dentre todos os caminhos existentes entre *v* e *w* .**





Caminho mais curto

1. De um nó fonte para todos os demais vértices

2. De todos os nós para um destino único

3. De par único

4. Dentre todos os pares de vértices

Arestas de Peso Negativo

Se o grafo G não contém ciclo de peso negativo, a partir da origem s, então para todo vértice $v \in V$, o comprimento do caminho mais curto $\delta(s,v)$ permanece bem definido, mesmo contendo arestas de peso negativo.

Contudo, se existe um ciclo de peso negativo a partir de s, os pesos de caminhos mais curtos não são bem definidos, pois sempre é possível encontrar um caminho ainda mais curto percorrendo o ciclo de peso negativo.

Caminho mais curto

***Um caminho mais curto entre dois vértices de um grafo pode conter um ciclo de peso positivo*?**

Caminho mais curto

***Um caminho mais curto entre dois vértices de um grafo pode conter um ciclo de peso positivo*?**

Exercício: Prove que um caminho mais curto entre dois vértices de um grafo não contém um ciclo de peso positivo.

Subestrutura ótima do caminho mais curto

Considere $p = \langle v_1, v_2, \ldots, v_k \rangle$, um caminho mais curto entre os vértices v_1 e v_k.

Então, para quaisquer i, j, tais que $1 \leq i \leq j \leq k$, o caminho $p_{ij} = \langle v_i, v_{i+1}, \ldots, vj \rangle$, subcaminho de p, é um caminho mais curto entre os vértices i e j.

Subestrutura ótima do caminho mais curto

Considere $p = \langle v_1, v_2, \ldots, v_k \rangle$, um caminho mais curto entre os vértices v_1 e v_k.

Então, para quaisquer i, j, tais que $1 \leq i \leq j \leq k$, o caminho $p_{ij} = \langle v_i, v_{i+1}, \ldots, vj \rangle$, subcaminho de p, é um caminho mais curto entre os vértices i e j.

Desigualdade Triangular

Para qualquer aresta $(u,v) \in E$, temos

$$\delta(s,v) \leq \delta(s,u) + w(u,v)$$

onde

$\delta(u,v)$ – caminho mais curto de u para v

$w(u,v)$ – peso da aresta (u,v)

Algoritmo de Dijkstra

1. De um nó fonte para todos os demais vértices

2. De todos os nós para um destino único

3. De par único

Algoritmo de Dijkstra

Dijkstra (G=(N,M,P))

$d_1 = 0$; rot (1) = ∞ // fonte = 1

$F := \{1\}$; $A = N \setminus \{1\}$

para j = 2 até n faça

  se $j \in \Gamma^+(1)$ então

    $d_j = p_{1j}$; rot [j] = 1

  senão

    $d_j = \infty$; rot [j] = 0

Enquanto $A \neq \emptyset$ faça

  $i \leftarrow \min \{d_j \mid j \in A\}$

  $A = A \setminus \{i\}$; $F = F \cup \{i\}$

  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça

    se $d_j > (d_i + p_{ij})$ então

      $d_j = d_i + p_{ij}$; rot [j] = i

Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞ // fonte = 1

F := {1}; *A* = *N* \ {1}

para *j* = 2 até *n* faça

se $j \in \Gamma^+(1)$ então

$d_j = p_{1j}$; rot [*j*] = 1

senão

$d_j = \infty$; rot [*j*] = 0

Enquanto $A \neq \emptyset$ faça

$i \leftarrow \min \{d_j \mid j \in A\}$

$A = A \setminus \{i\}$; $F = F \cup \{i\}$

para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça

se $d_j > (d_i + p_{ij})$ então

$d_j = d_i + p_{ij}$; rot [*j*] = *i*

Lista de nós **F**echados e **A**bertos

Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞ // fonte = 1

F := {1}; *A* = *N* \ {1}

para *j* = 2 até *n* faça

se $j \in \Gamma^+(1)$ então

$d_j = p_{1j}$; rot [*j*] = 1

senão

$d_j = \infty$; rot [*j*] = 0

Enquanto $A \neq \emptyset$ faça

$i \leftarrow \min \{d_j \mid j \in A\}$

$A = A \setminus \{i\}$; $F = F \cup \{i\}$

para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça

se $d_j > (d_i + p_{ij})$ então

$d_j = d_i + p_{ij}$; rot [*j*] = *i*

Inicia os rótulos dos vértices

Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞ // fonte = 1

F := {1}; *A* = *N* \ {1}

para *j* = 2 até *n* faça

se $j \in \Gamma^+(1)$ então

$d_j = p_{1j}$; rot [*j*] = 1

senão

$d_j = \infty$; rot [*j*] = 0

Enquanto $A \neq \emptyset$ faça

$i \leftarrow \min \{d_j \mid j \in A\}$

$A = A \setminus \{i\}$; $F = F \cup \{i\}$

para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça

se $d_j > (d_i + p_{ij})$ então

$d_j = d_i + p_{ij}$; rot [*j*] = *i*

Calcula o caminho mais curto do vértice 1 para os outros vértices do grafo

Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞

$F := \{1\}$; $A = N \setminus \{1\}$

para $j = 2$ até n faça

  se $j \in \Gamma^+(1)$ então

    $d_j = p_{1j}$; rot [j] = 1

  senão

    $d_j = \infty$; rot [j] = 0

Enquanto $A \neq \emptyset$ faça

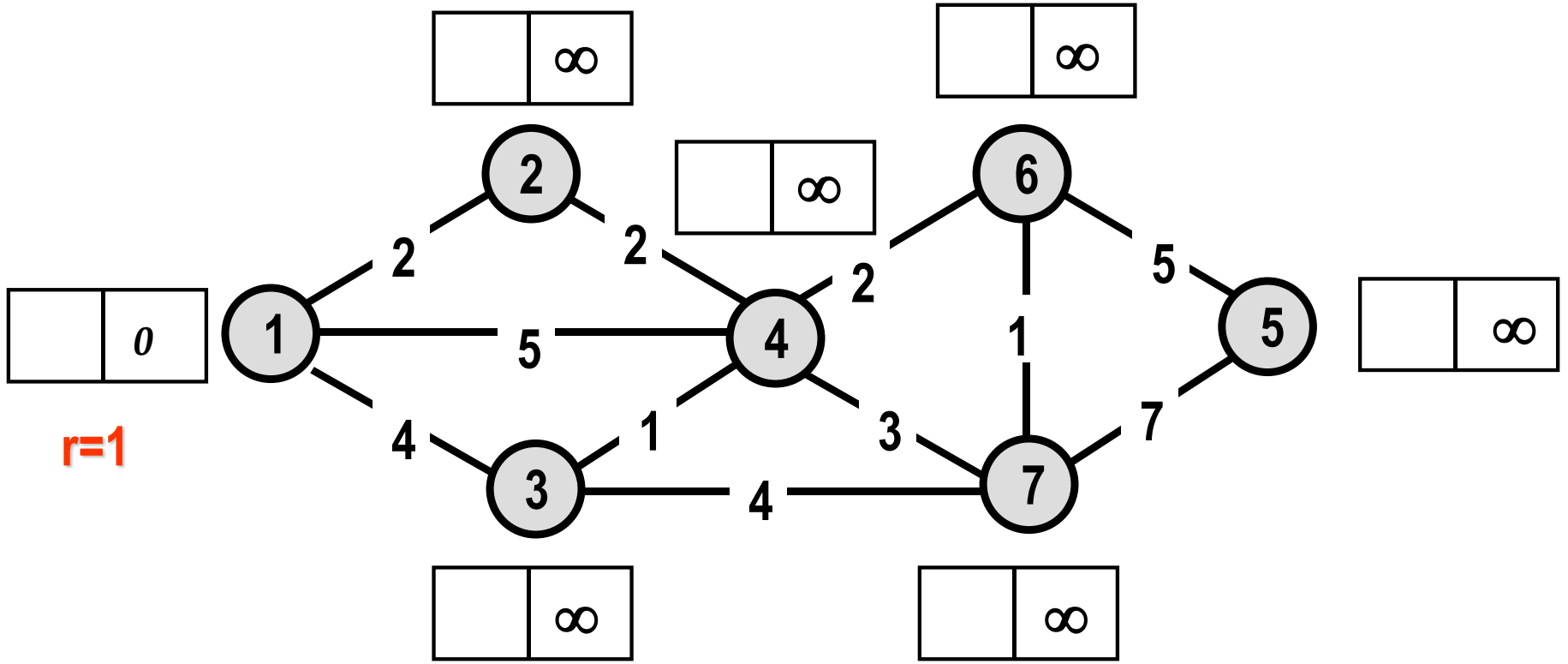
  $i \leftarrow \min \{d_j \mid j \in A\}$

  $A = A \setminus \{i\}$; $F = F \cup \{i\}$

  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça

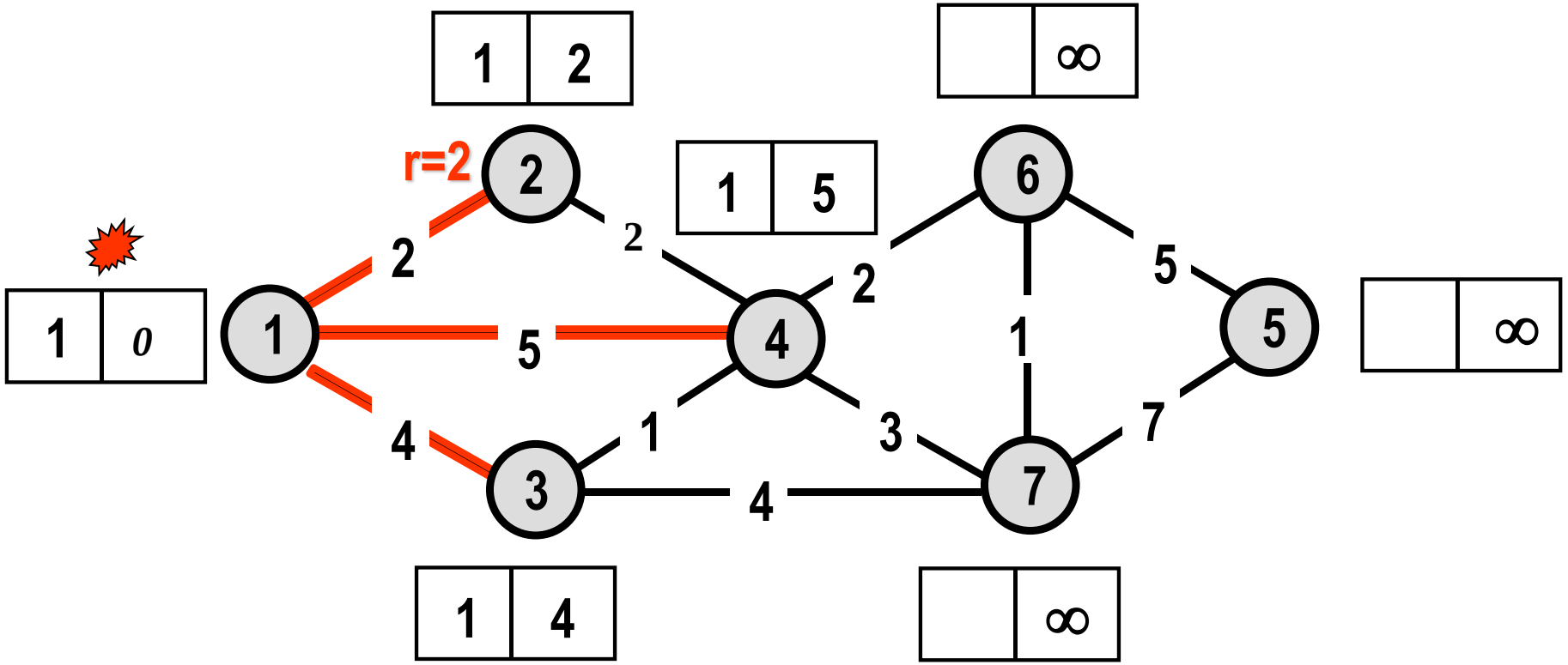
    se $d_j > (d_i + p_{ij})$ então

      $d_j = d_i + p_{ij}$; rot [j] = i



Algoritmo de Dijkstra

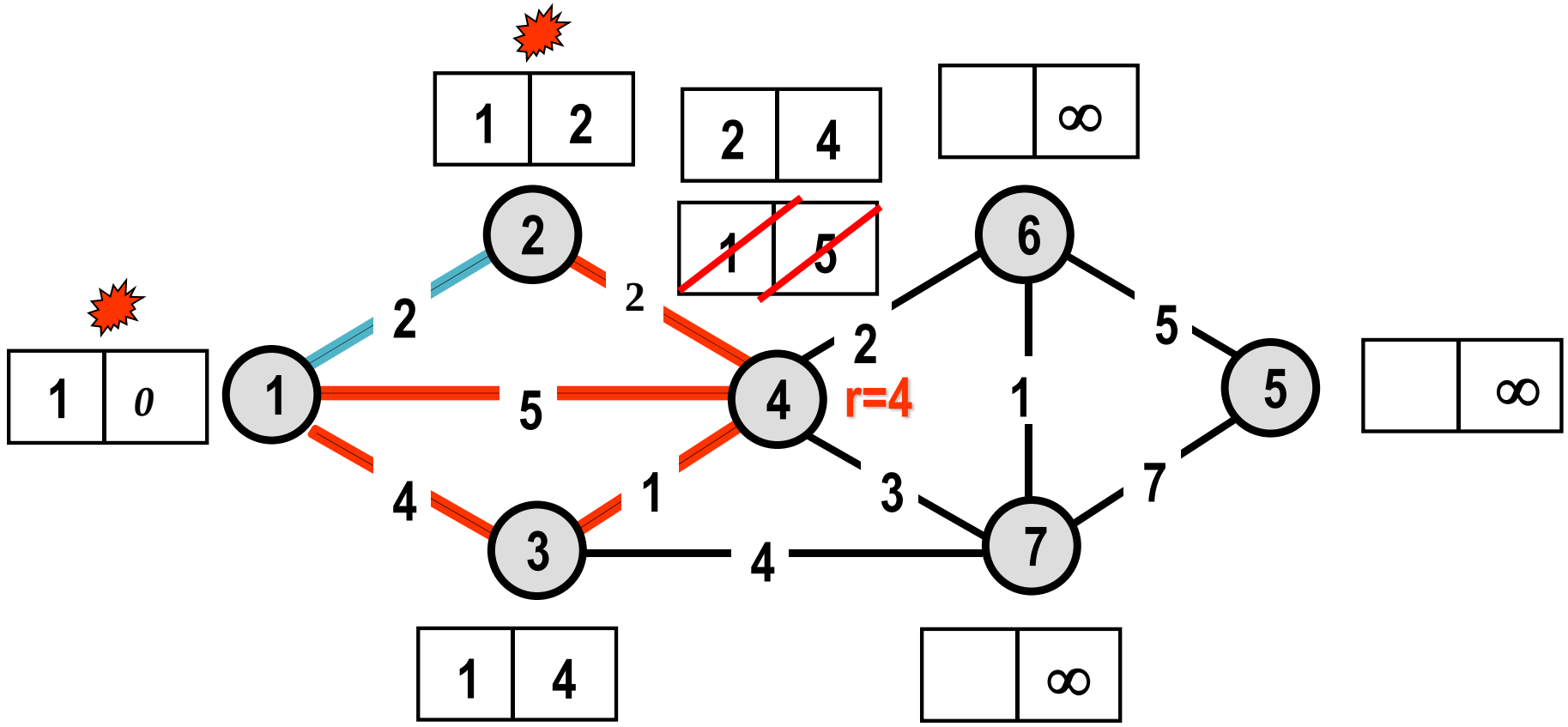
```
Dijkstra (G=(N,M,P) )
    d_1 = 0; rot (1) = ∞
    F := {1}; A = N \ {1}
    para j = 2 até n faça
      se j ∈ Γ+(1) então
        d_j = p_1j ; rot [j] = 1
      senão
        d_j = ∞;  rot [j] = 0
    Enquanto A ≠ ∅ faça
      i ← min {d_j | j ∈ A}
      A = A \ {i} ;   F = F ∪ {i}
      para todo ( j ∈ Γ+(i ) e ( j ∈ A) faça
        se d_j > (d_i + p_ij) então
          d_j = d_i + p_ij;  rot [j] = i
```



Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

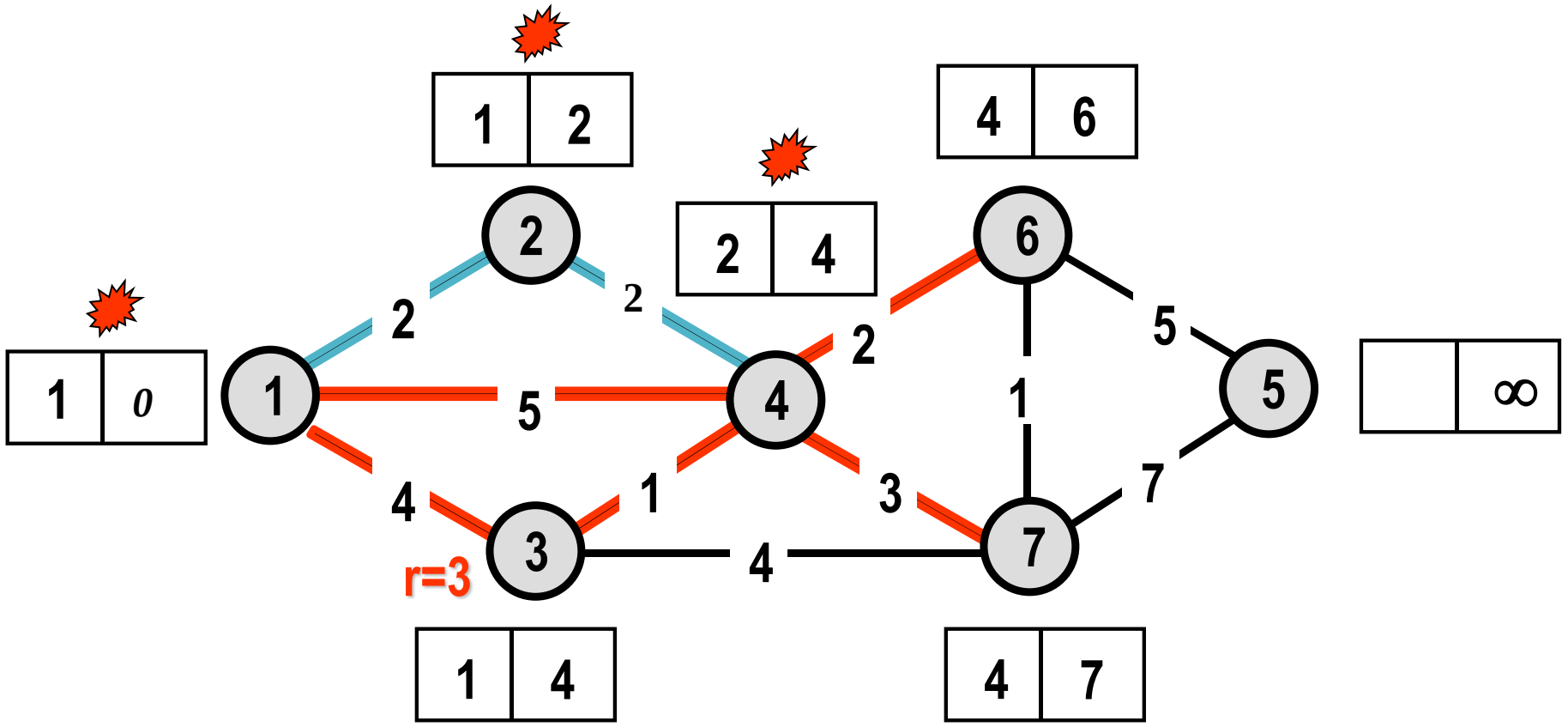
$d_1 = 0$; rot (1) = ∞
F := {1}; *A* = *N* \ {1}
para *j* = 2 até *n* faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [*j*] = 1
  senão
    $d_j = \infty$; rot [*j*] = 0
Enquanto $A \neq \emptyset$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [*j*] = *i*



Algoritmo de Dijkstra

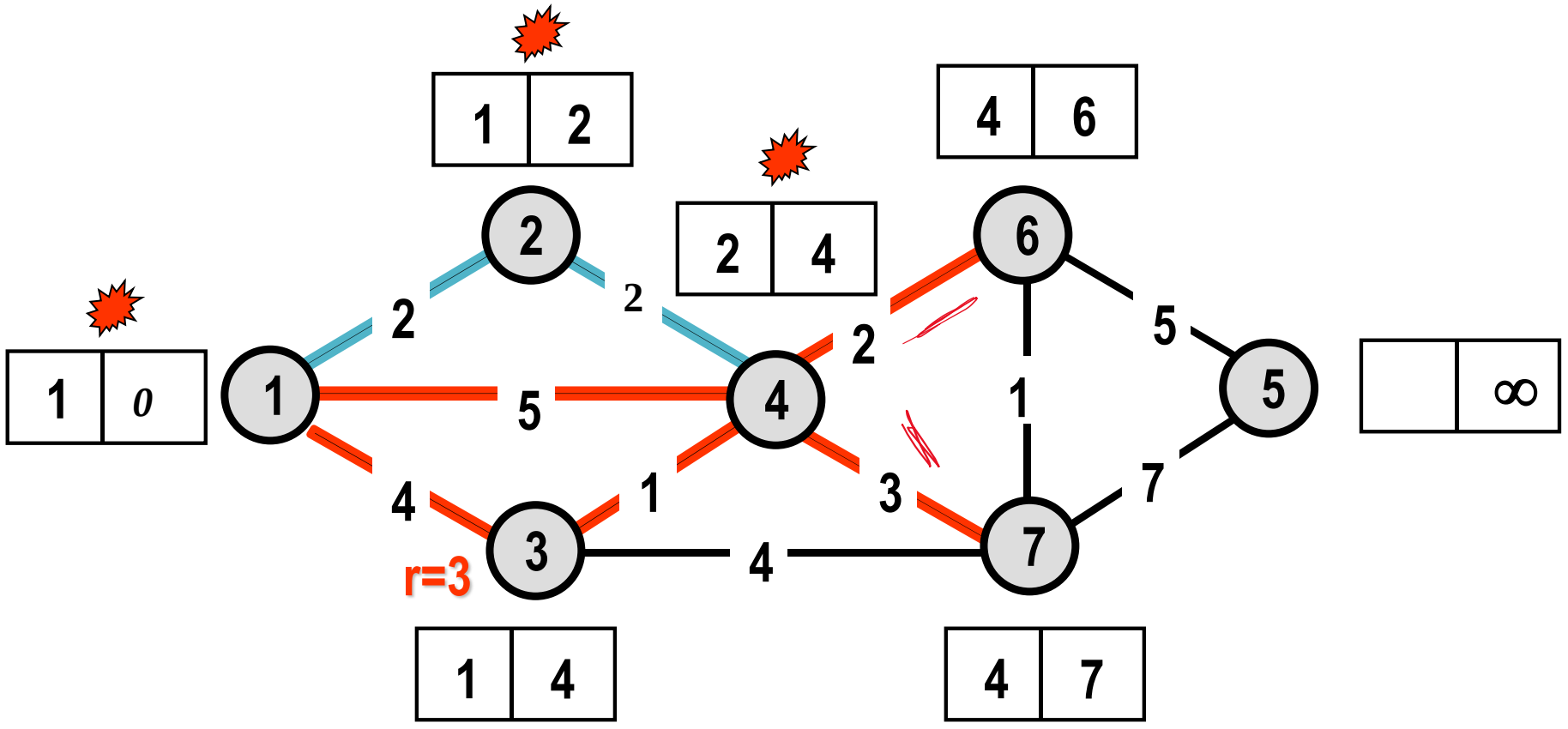
Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞
$F := \{1\}$; $A = N \setminus \{1\}$
para j = 2 até n faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [j] = 1
  senão
    $d_j = \infty$; rot [j] = 0
Enquanto $A \neq \emptyset$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [j] = i



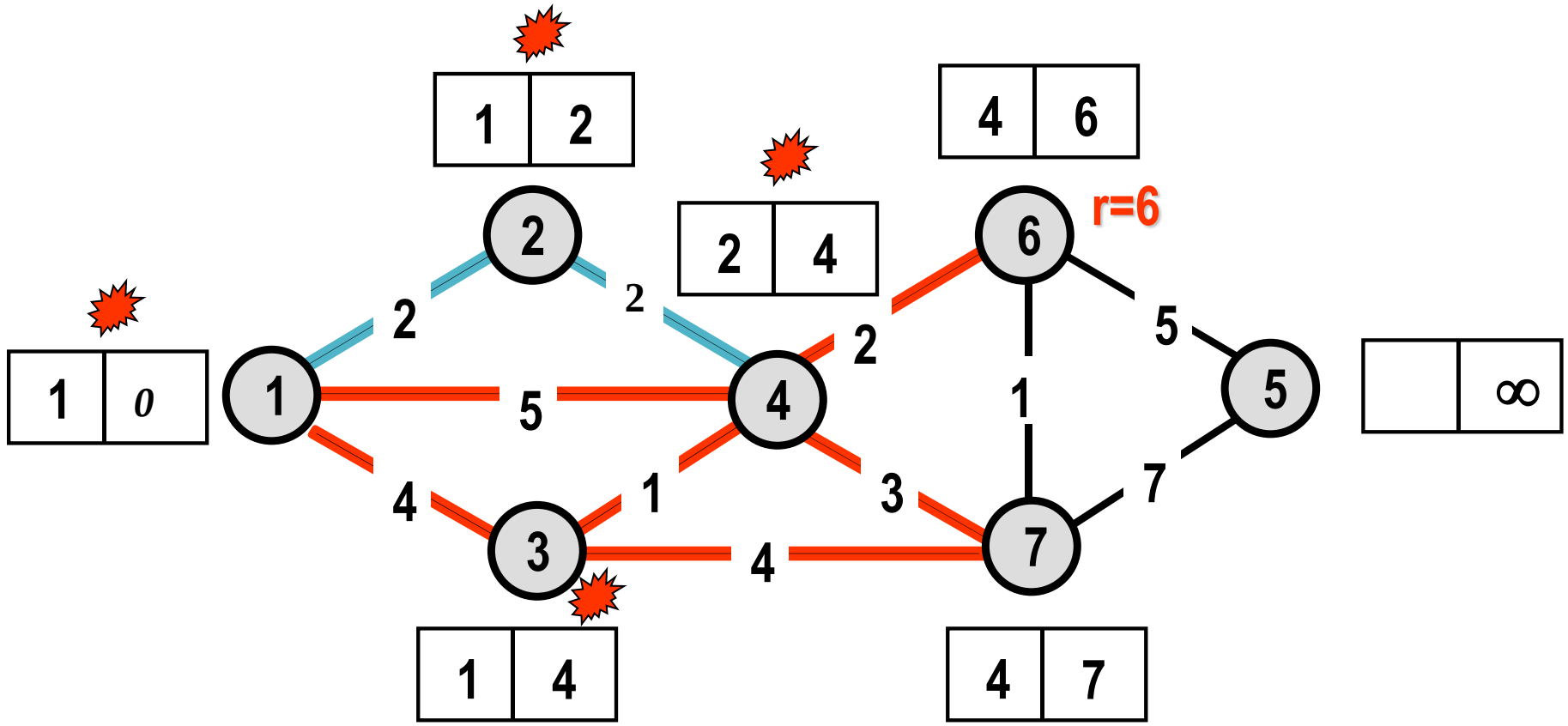
Algoritmo de Dijkstra

```
Dijkstra (G=(N,M,P) )
    d_1 = 0; rot (1) = ∞
    F := {1}; A = N \ {1}
    para j = 2 até n faça
       se j ∈ Γ+(1) então
          d_j = p_1j ; rot [j] = 1
       senão
          d_j = ∞;  rot [j] = 0
    Enquanto A ≠ ∅ faça
       i ← min {d_j | j ∈ A}
       A = A \ {i} ;   F = F ∪ {i}
       para todo ( j ∈ Γ+(i ) e ( j ∈ A) faça
          se d_j > (d_i + p_ij) então
             d_j = d_i + p_ij;  rot [j] = i
```



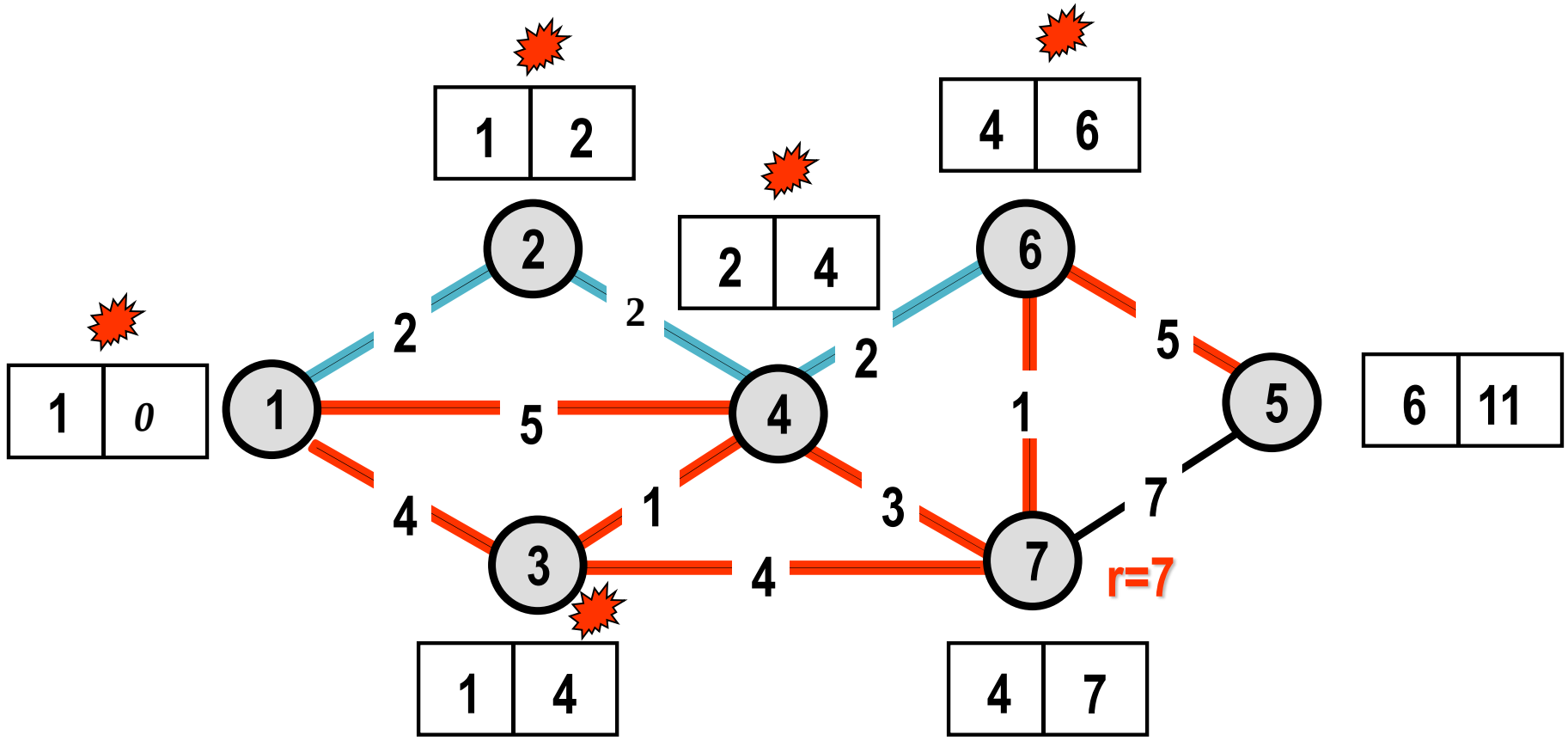
Algoritmo de Dijkstra

```
Dijkstra (G=(N,M,P) )
    d_1 = 0; rot (1) = ∞
    F := {1}; A = N \ {1}
    para j = 2 até n faça
       se j ∈ Γ+(1) então
          d_j = p_1j ; rot [j] = 1
       senão
          d_j = ∞;  rot [j] = 0
    Enquanto A ≠ ∅ faça
       i ← min {d_j | j ∈ A}
       A = A \ {i} ;   F = F ∪ {i}
       para todo ( j ∈ Γ+(i ) e ( j ∈ A) faça
          se d_j > (d_i + p_ij) então
             d_j = d_i + p_ij;  rot [j] = i
```



Algoritmo de Dijkstra

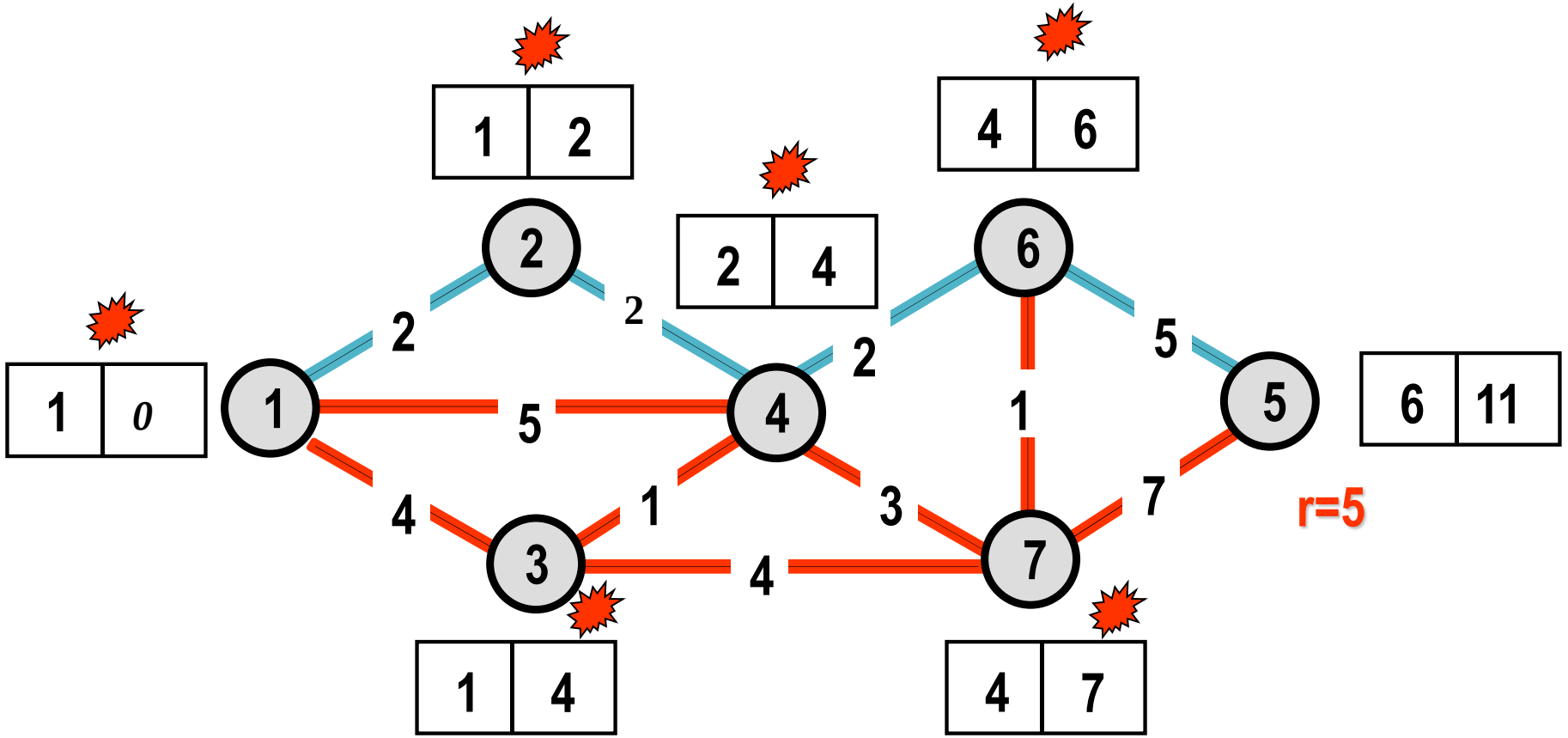
```
Dijkstra (G=(N,M,P) )
    d_1 = 0; rot (1) = ∞
    F := {1}; A = N \ {1}
    para j = 2 até n faça
       se j ∈ Γ+(1) então
          d_j = p_1j ; rot [j] = 1
       senão
          d_j = ∞;  rot [j] = 0
    Enquanto A ≠ ∅ faça
       i ← min {d_j | j ∈ A}
       A = A \ {i} ;   F = F ∪ {i}
       para todo ( j ∈ Γ+(i ) e ( j ∈ A) faça
          se d_j > (d_i + p_ij) então
             d_j = d_i + p_ij;  rot [j] = i
```



Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

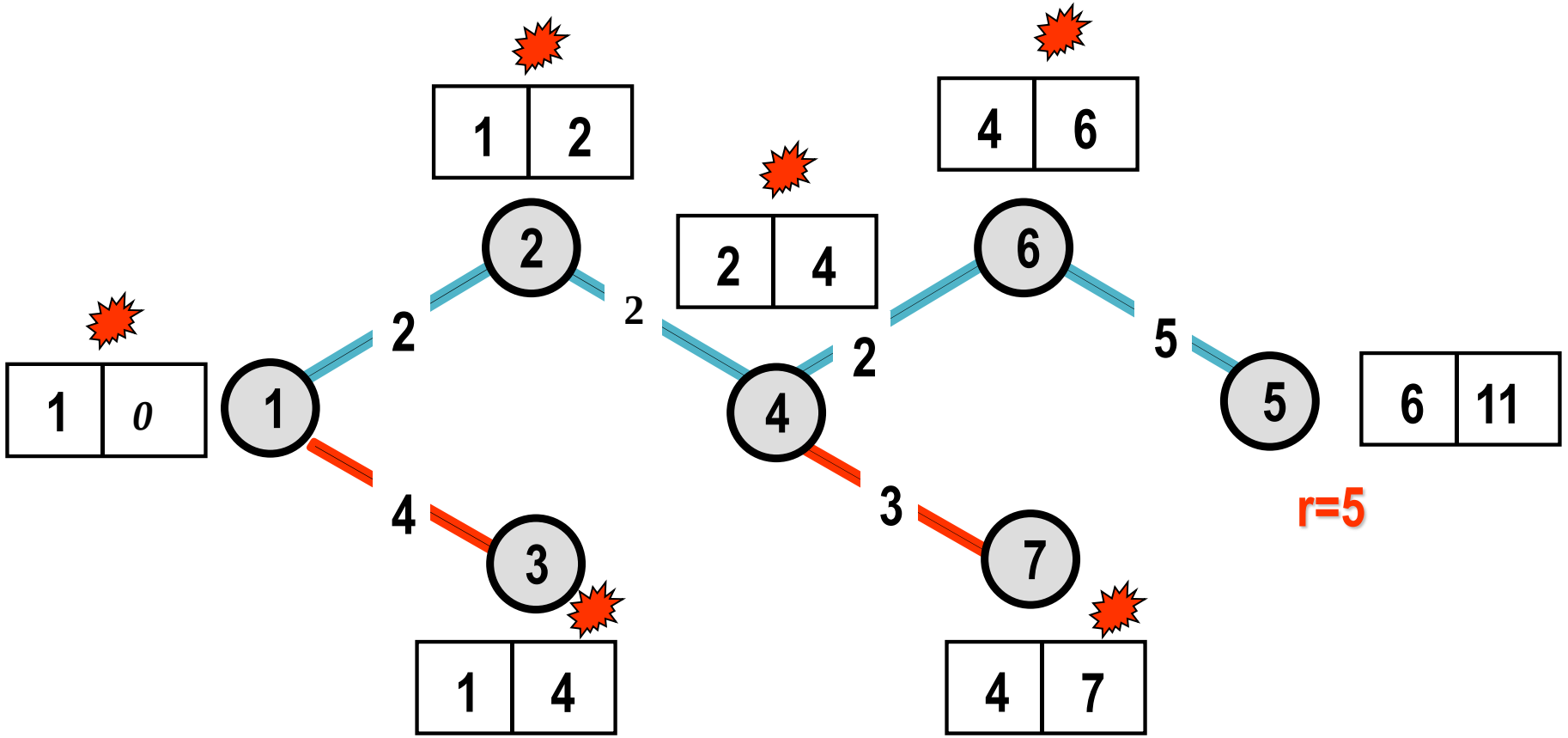
$d_1 = 0$; rot (1) = ∞
$F := \{1\}$; $A = N \setminus \{1\}$
para j = 2 até n faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [j] = 1
  senão
    $d_j = \infty$; rot [j] = 0
Enquanto $A \neq \emptyset$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [j] = i



Algoritmo de Dijkstra

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞
F := {1}; *A* = *N* \ {1}
para *j* = 2 até *n* faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [*j*] = 1
  senão
    $d_j = \infty$; rot [*j*] = 0
Enquanto $A \neq \varnothing$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [*j*] = *i*



Algoritmo de Dijkstra - Correto

O algoritmo de Dijkstra encontra o caminho mais curto de um vértice fonte aos demais vértices do grafo.

Algoritmo de Dijkstra - Correto

O algoritmo de Dijkstra encontra o caminho mais curto de um vértice fonte aos demais vértices do grafo.

Prova:

A demonstração de que o algoritmo de Dijkstra está correto será feita por indução em *F*, considerando que se o nó *i* está em *F*, então *d*[*i*] é o caminho mais curto da fonte até o nó *i*.

Algoritmo de Dijkstra - Correto

Base da Indução:

No início *F* só contém o nó 1 (fonte) e o teorema vale trivialmente.

Hipótese da Indução:

A hipótese vale para todos os nós de *F* até imediatamente antes da inserção de um nó *i*.

Algoritmo de Dijkstra - Correto

Passo da Indução:

Se o nó *i* foi escolhido pelo algoritmo, então *dt*[*i*] é mínimo dentre todos os nós em *A*.

Deve-se mostrar que *dt*[*i*] é o comprimento do menor caminho entre o nó origem e *i*.

Supor o contrário, ou seja, que existe pelo menos um nó *x* no menor caminho entre a origem e *i*, que não pertence ao caminho atual de comprimento *dt*[*i*], tal que *dt*[*x*] < *dt*[*i*] e *x* está em *A*.

Nesse caso, o algoritmo deveria ter escolhido *x* ao invés de *i*. Mas o algoritmo escolheu *i*, significando que este nó *x* não existe. Portanto, quando *i* é adicionado a *F* o caminho mais curto da origem a *i* foi encontrado.

■

Algoritmo de Dijkstra - Complexidade

Após a inicialização, o algoritmo de Dijkstra executa dois tipos de ação em cada uma das suas iterações:

(1) Determina o próximo vértice a ser examinado

(2) Atualiza os rótulos dos vértices que se ligam ao vértice correntemente examinado

Algoritmo de Dijkstra - Complexidade

Para que *F* contenha todos os vértices do grafo, são necessárias *n*-1 iterações do "enquanto".

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞
$F := \{1\}$; $A = N \setminus \{1\}$
para $j = 2$ até n faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [j] = 1
  senão
    $d_j = \infty$; rot [j] = 0
Enquanto $A \neq \varnothing$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [j] = i

Algoritmo de Dijkstra - Complexidade

Para que *F* contenha todos os vértices do grafo, são necessárias *n*-1 iterações do "enquanto".

Sem utilizar estruturas de dados especiais, as ações do tipo (1), utilizando uma busca em lista linear, devem percorrer todos os vértices do conjunto *A* no pior caso, consumindo $O(n)$ comparações.

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞
$F := \{1\}$; $A = N \setminus \{1\}$
para $j = 2$ até n faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [j] = 1
  senão
    $d_j = \infty$; rot [j] = 0
Enquanto $A \neq \emptyset$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$ (1)
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [j] = i

Algoritmo de Dijkstra - Complexidade

Para que *F* contenha todos os vértices do grafo, são necessárias *n*-1 iterações do "enquanto".

Sem utilizar estruturas de dados especiais, as ações do tipo (1), utilizando uma busca em lista linear, devem percorrer todos os vértices do conjunto *A* no pior caso, consumindo $O(n)$ comparações.

As ações do tipo (2) podem envolver a atualização de $O(n)$ vértices.

Assim, uma implementação simples do o algoritmo de Dijkstra tem complexidade $O(n^2)$.

Dijkstra (G=(*N*,*M*,*P*))

$d_1 = 0$; rot (1) = ∞
$F := \{1\}$; $A = N \setminus \{1\}$
para $j = 2$ até n faça
  se $j \in \Gamma^+(1)$ então
    $d_j = p_{1j}$; rot [*j*] = 1
  senão
    $d_j = \infty$; rot [*j*] = 0
Enquanto $A \neq \emptyset$ faça
  $i \leftarrow \min \{d_j \mid j \in A\}$
  $A = A \setminus \{i\}$; $F = F \cup \{i\}$
  para todo ($j \in \Gamma^+(i)$ e ($j \in A$) faça
    se $d_j > (d_i + p_{ij})$ então
      $d_j = d_i + p_{ij}$; rot [*j*] = *i*

2

Algoritmo de Dijkstra

Arestas com custo negativo

Como o algoritmo emprega um critério guloso para fechar a cada iteração o vértice aberto e de menor caminho acumulado, torna-se incapaz de calcular o caminho mais curto em grafos com arestas de custo negativo.

Algoritmo de Dijkstra

Caminho mais curto entre todos os pares de vértices

O algoritmo pode ser utilizado para encontrar o caminho mais curto entre todos os pares de vértices do grafo desde que seja executado uma vez para cada vértice como origem.

Nessa situação a complexidade do método seria multiplicada por n, tornando-se $O(n^3)$ na implementação sem estruturas de dados especiais.

Algoritmo Dijkstra - Implementações

Ano	Autores	Complexidade
1987	Fredman e Tarjan	$O(m + n\log n)$
1990	Ahuja *et al.*	$O(m + n\sqrt{\log c})$
1996	Raman	$O(m + n\sqrt{\log n})$
1997	Raman	$O(m + n(\log c \log\log c)^{1/3})$
1999	Thorup	$O(m)$ - grafo não direcionado
2000	Hagerup	$O(m \log\log c)$
2000	Thorup	$O(m \log\log n)$

Obs. A constante *c* é um limite superior para o peso das arestas

Algoritmo de Bellman-Ford

Permite arestas de peso negativo.

Detecta ciclo negativo, caso exista

Algoritmo de Bellman-Ford

A ideia geral do algoritmo consiste em saber que um caminho direcionado de *s* para *j* contendo, no máximo, *k*+1 arcos pode ser obtido daqueles de *s* para *j* contendo *k* arcos.

As *n*-1 iterações do algoritmo refletem um caminho com, no máximo, *n*-1 arcos.

Algoritmo de Bellman-Ford

Verifica:

$$l_j^{(k+1)} = \min\{l_j^{(k)}, min_i[l_i^k, l(i,j)]\} \quad i \neq j, s; j = 1, \ldots, n-1$$

onde

$l_j^{(k)}$ Denota o peso do caminho entre *s* e *j* considerando *k* arestas.

$l(i,j)$ Denota o peso da aresta *(i,j)*.

Algoritmo de Bellman-Ford

```
Dados G=(V,E), s
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Algoritmo de Bellman-Ford

```
Dados G=(V,E), s
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Inicia rótulos

Algoritmo de Bellman-Ford

```
Dados G=(V,E), s
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Distância de s para s

Algoritmo de Bellman-Ford

```
Dados G=(V,E), s
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Calcula distância de s para todos os vértices do grafo

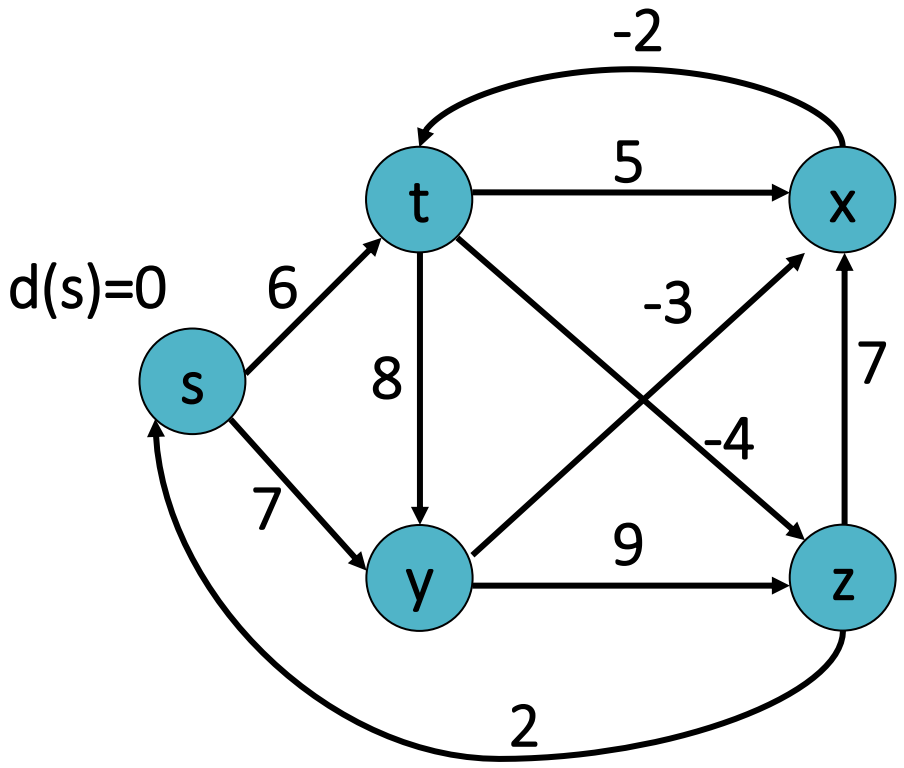
Algoritmo de Bellman-Ford

```
Dados G=(V,E), s
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Verifica a existência de ciclo com peso negativo

Algoritmo de Bellman-Ford – Exemplo 1

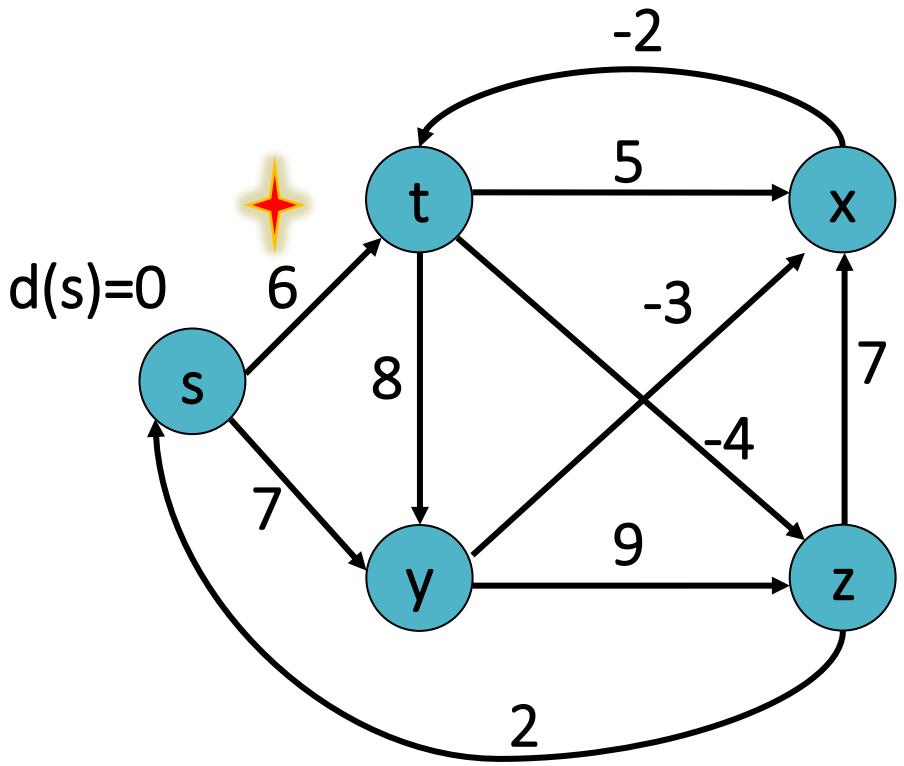
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	∞	NIL
x	∞	NIL
y	∞	NIL
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 1

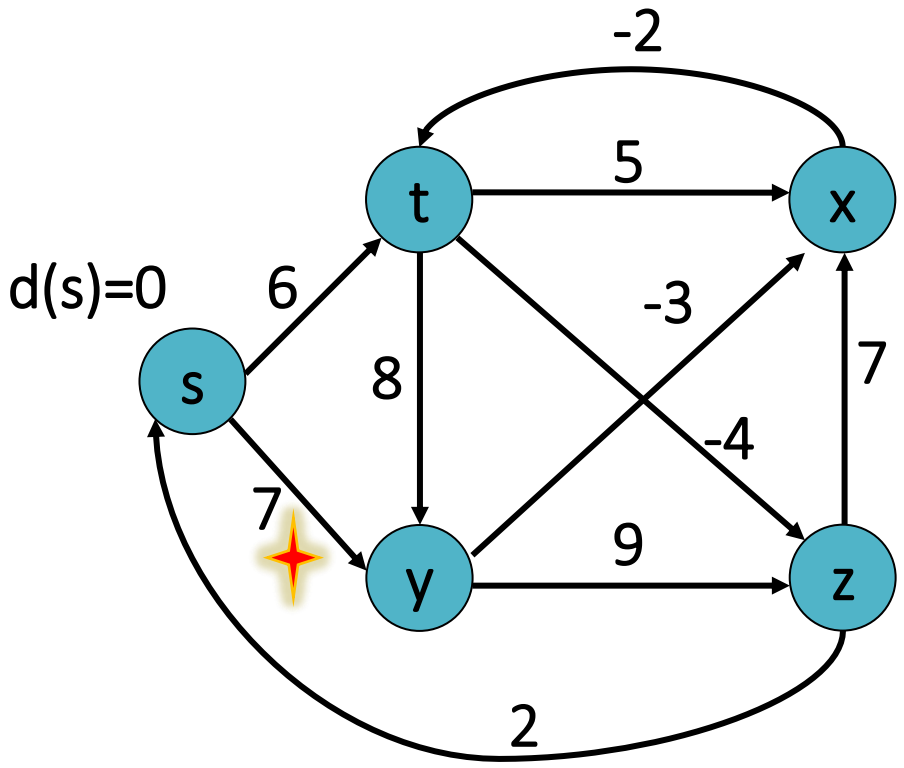
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	∞	NIL
y	∞	NIL
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 1

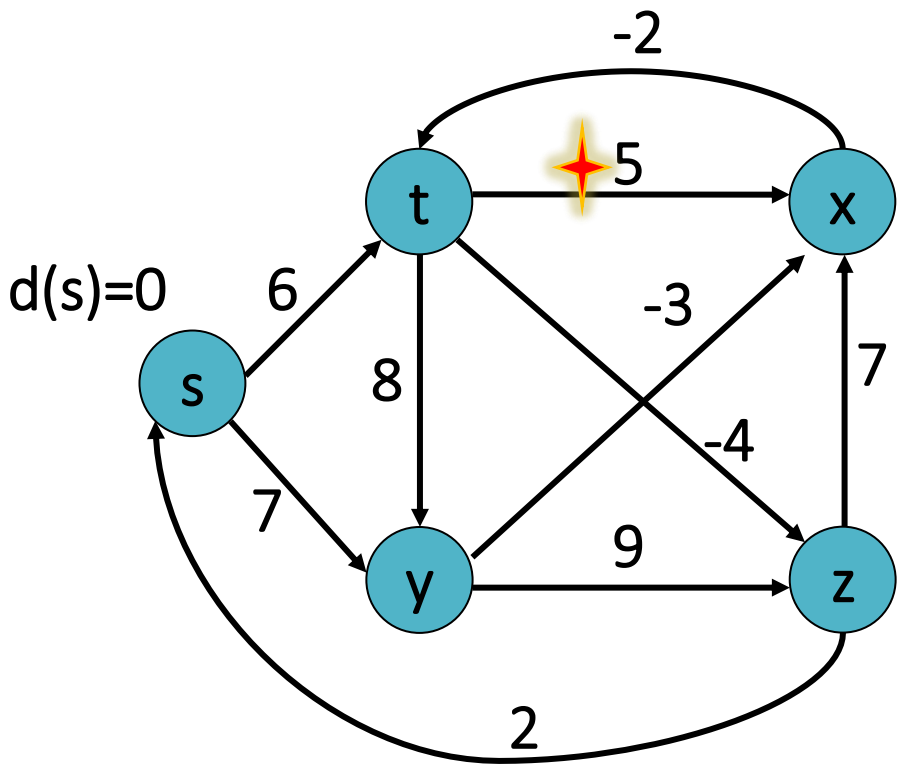
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	∞	NIL
y	7	s
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 1

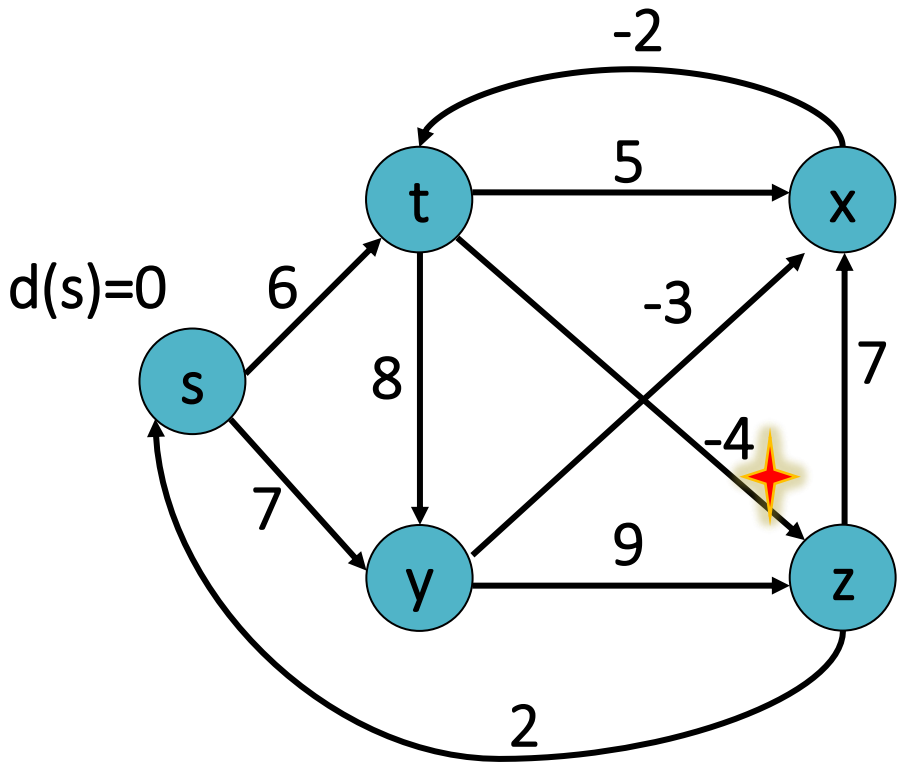
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	11	t
y	7	s
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 1

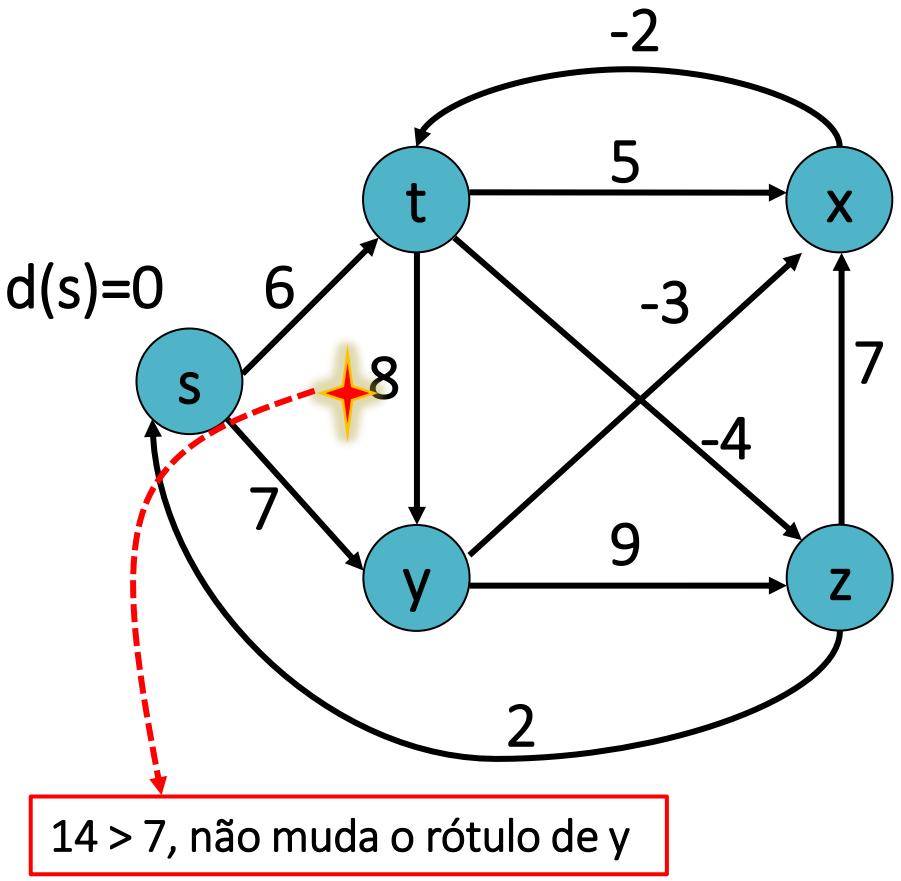
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	11	t
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

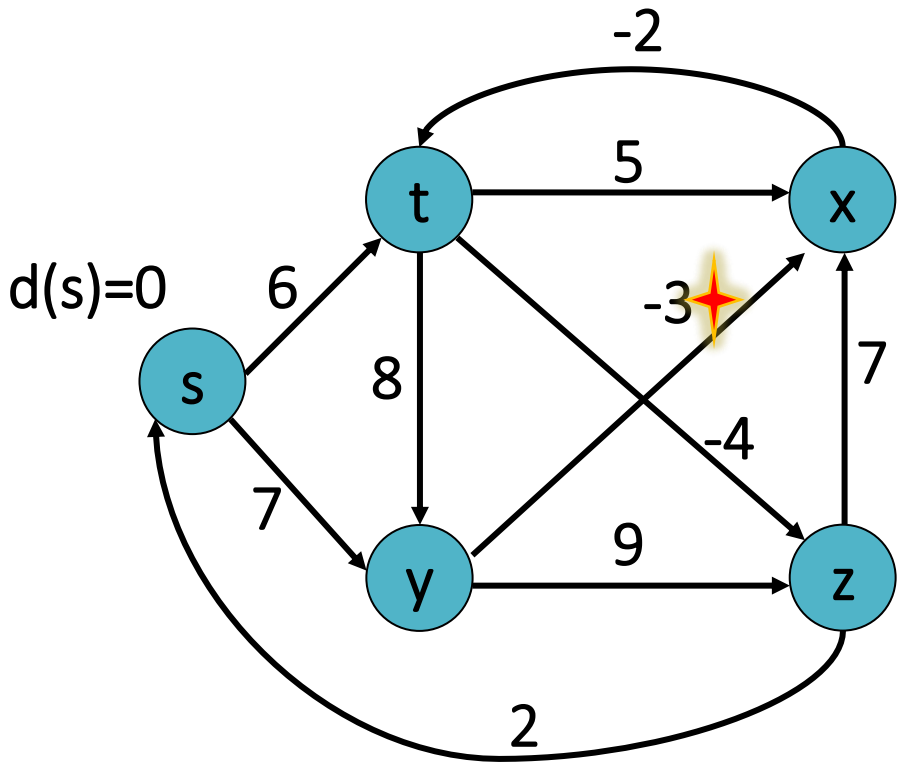
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	11	t
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

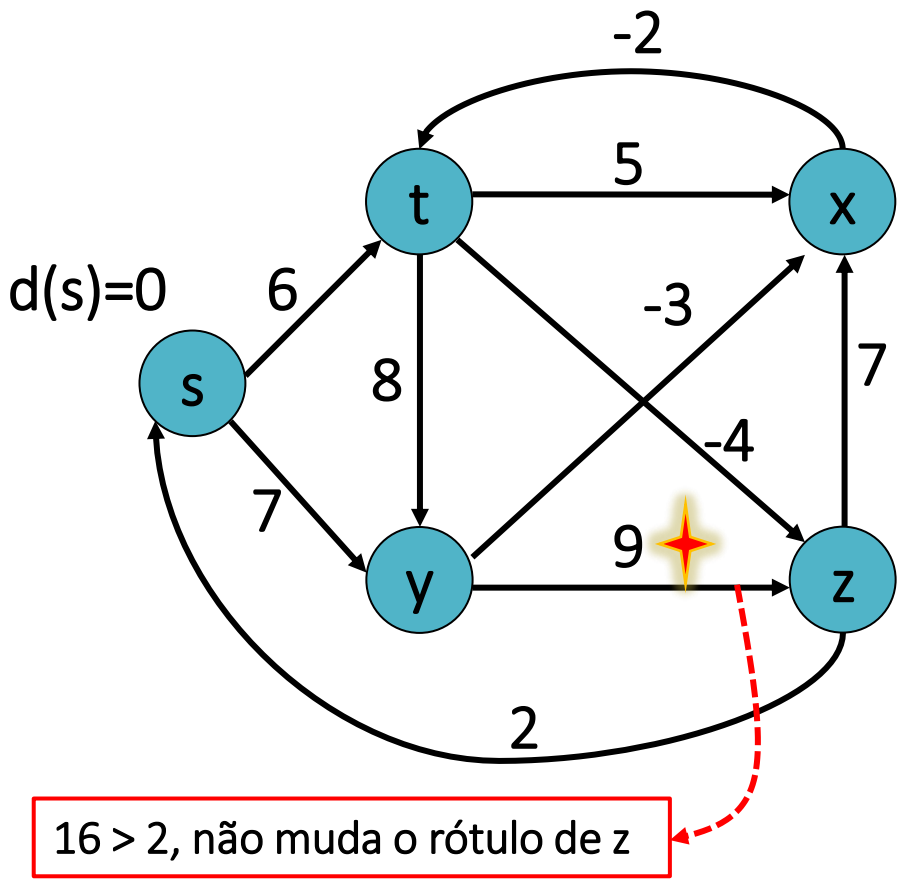
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

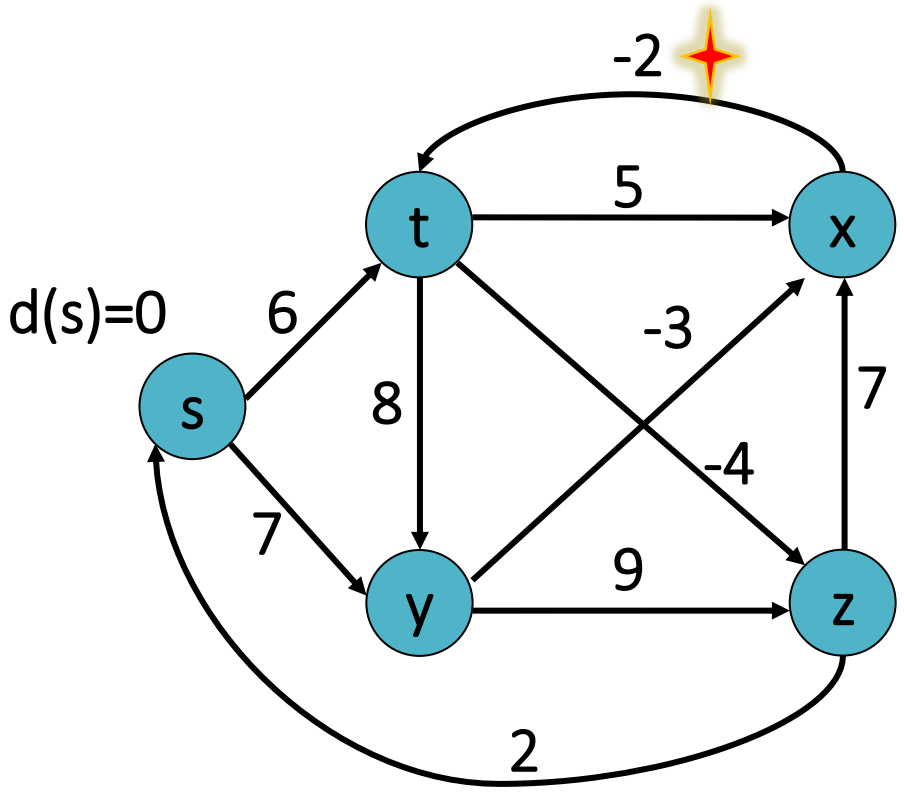
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	6	s
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

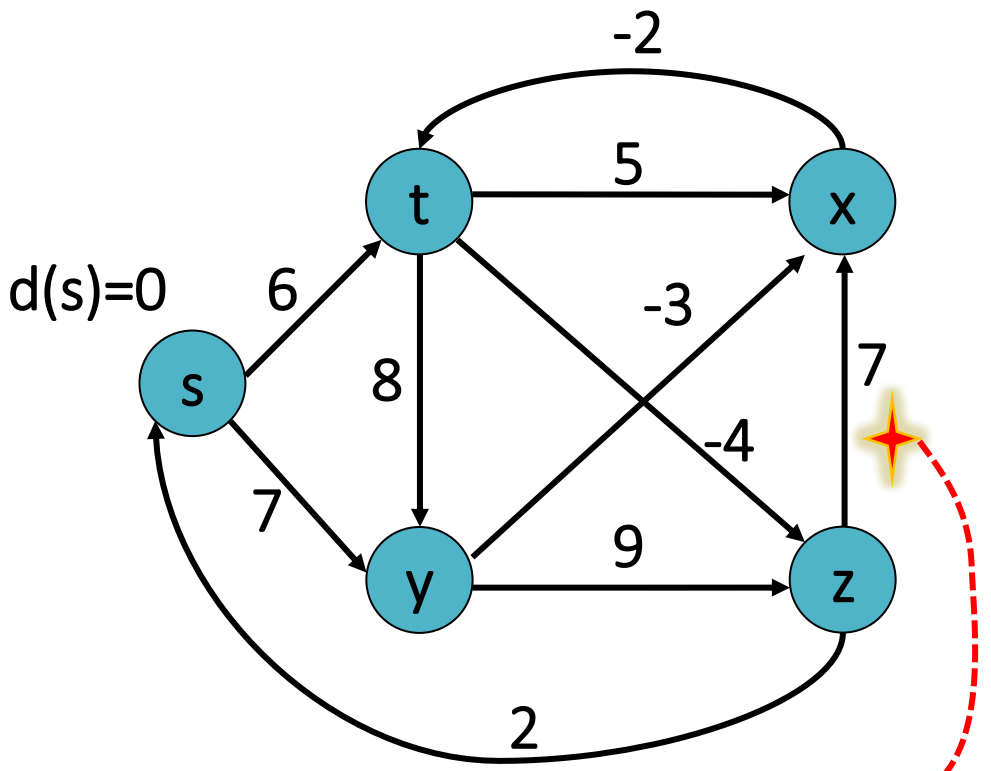
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

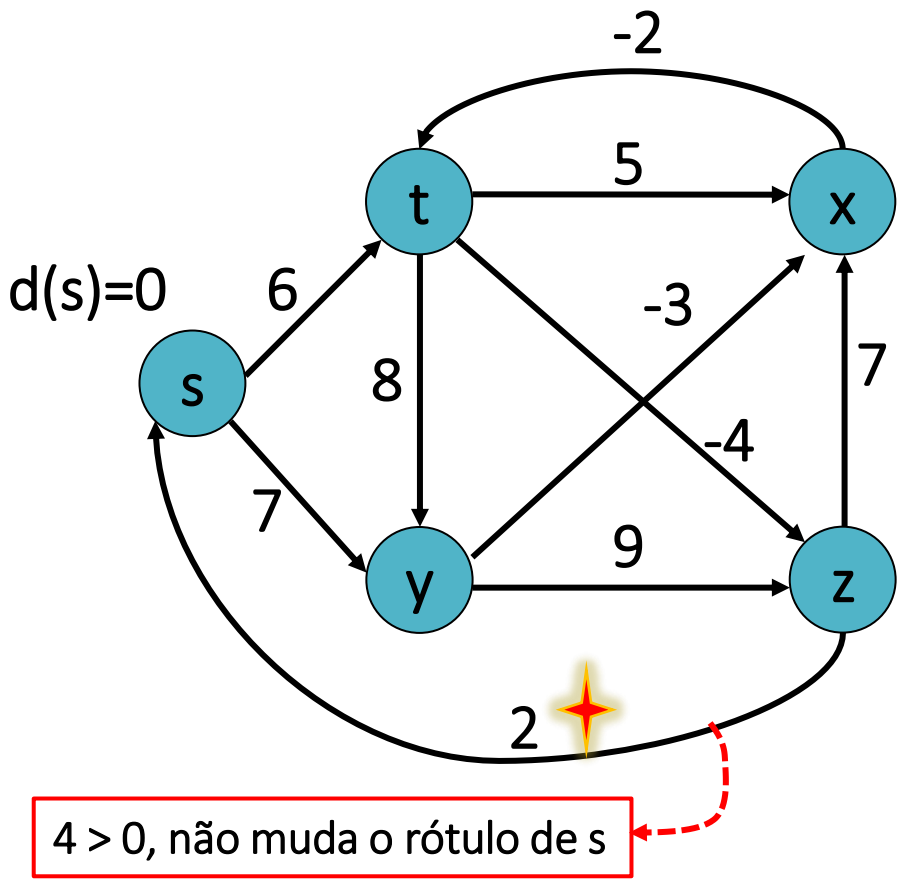
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



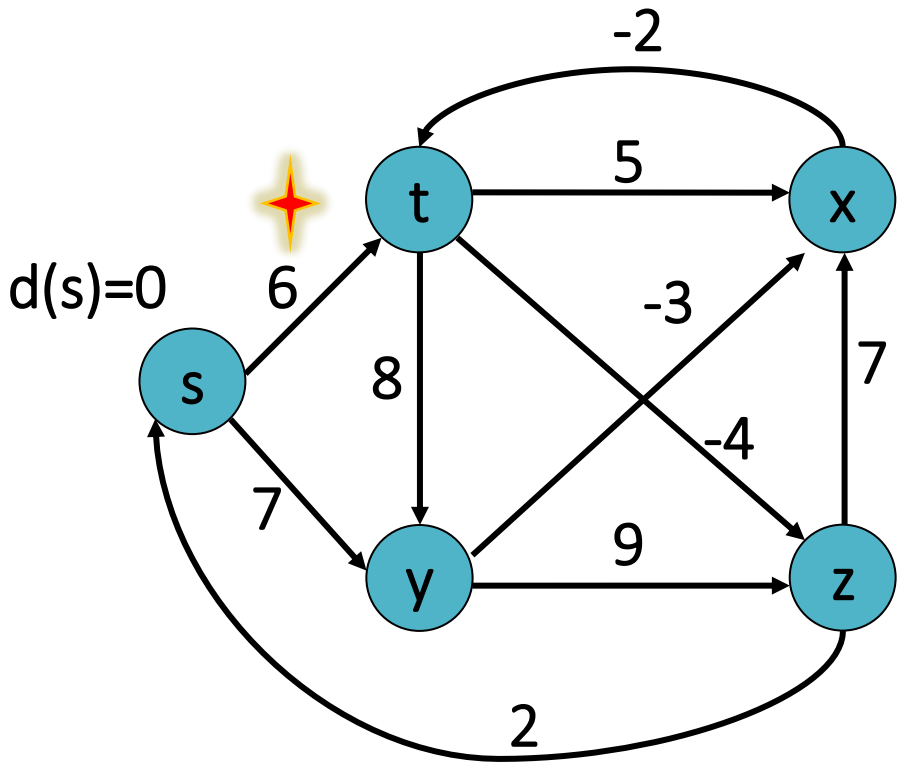
	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

Nova execução do laço

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

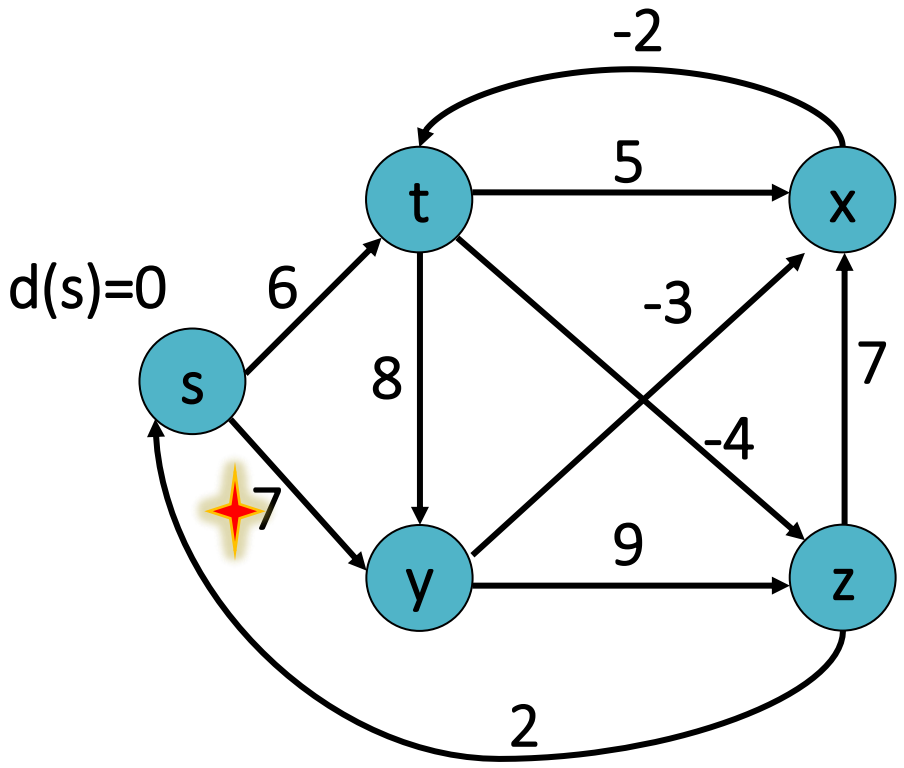


6 > 2, não muda o rótulo de t

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

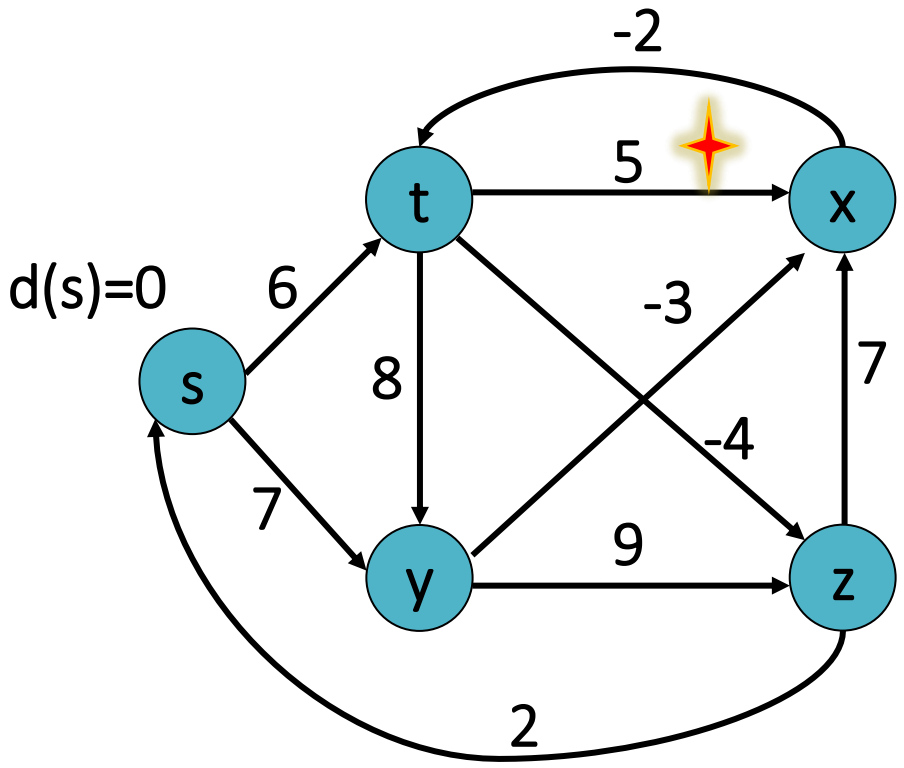


7 = 7, não muda o rótulo de y

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

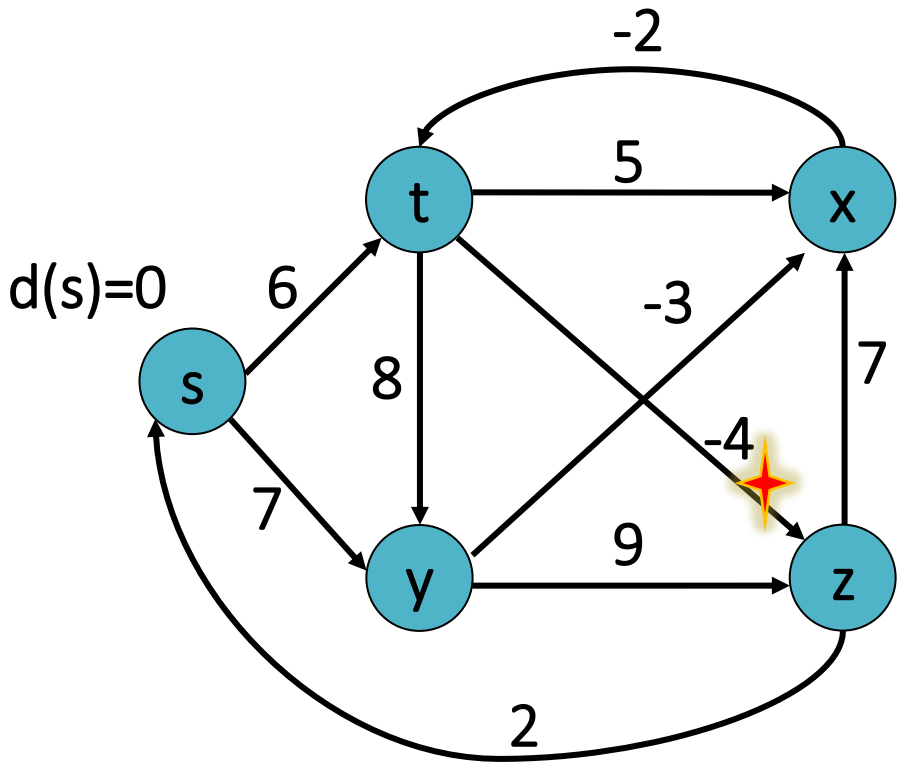


7 > 4, não muda o rótulo de x

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	2	t

Algoritmo de Bellman-Ford – Exemplo 1

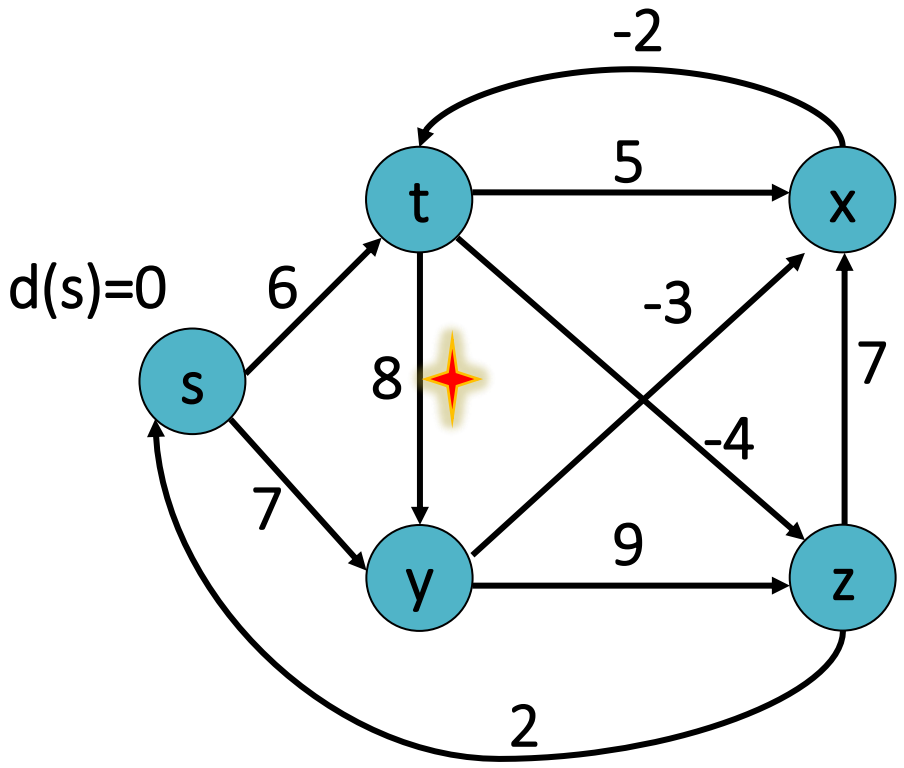
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

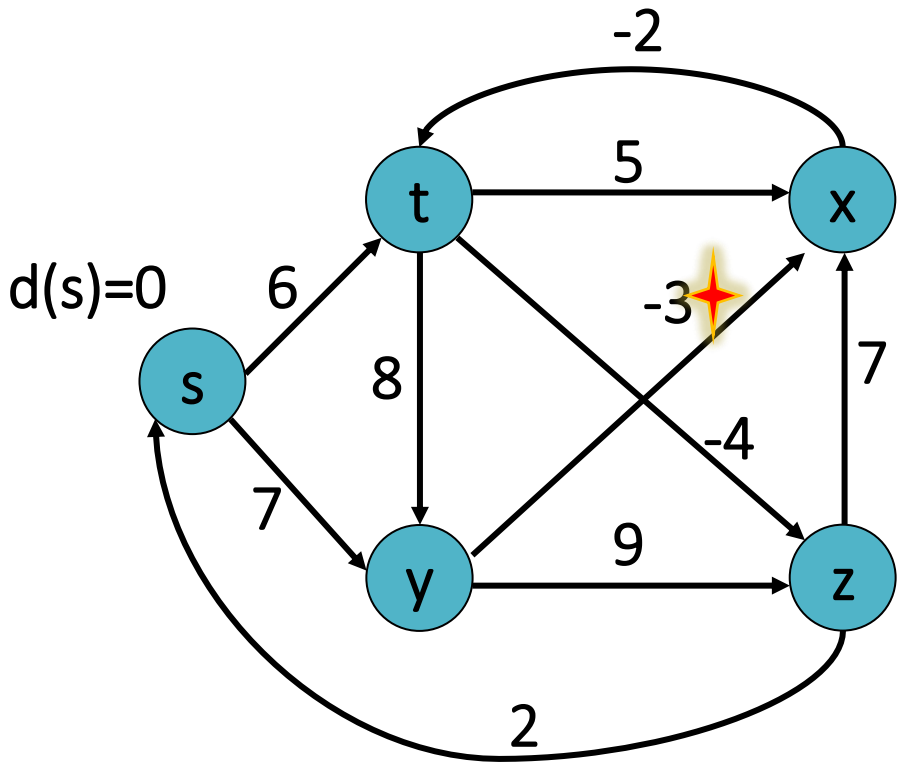


10 > 7 , não muda o rótulo de y

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

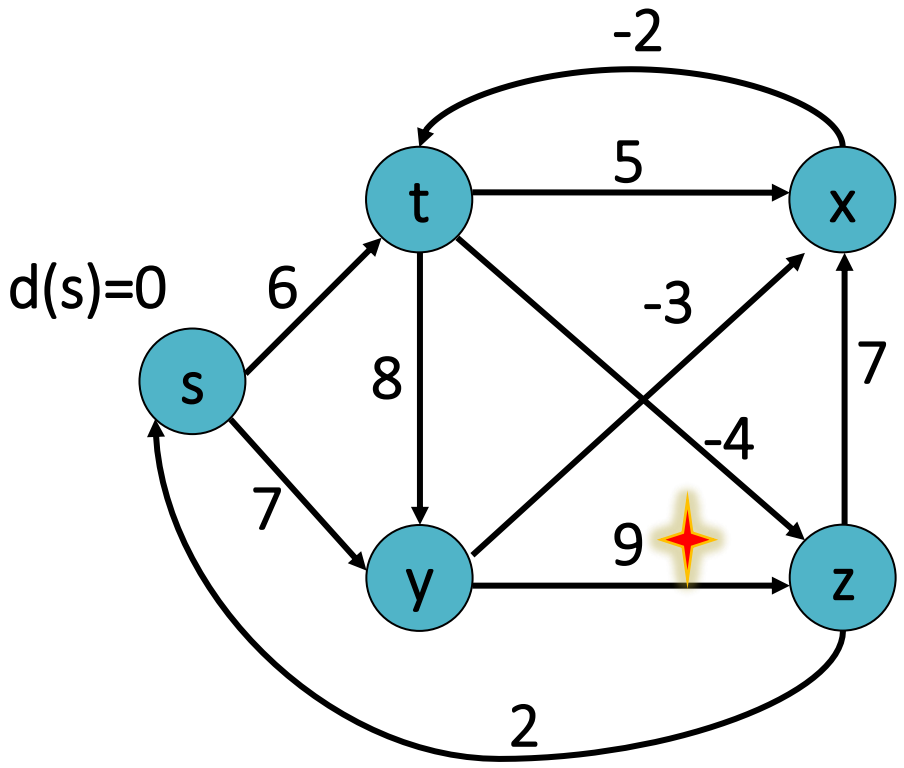


4 = 4 , não muda o rótulo de x

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

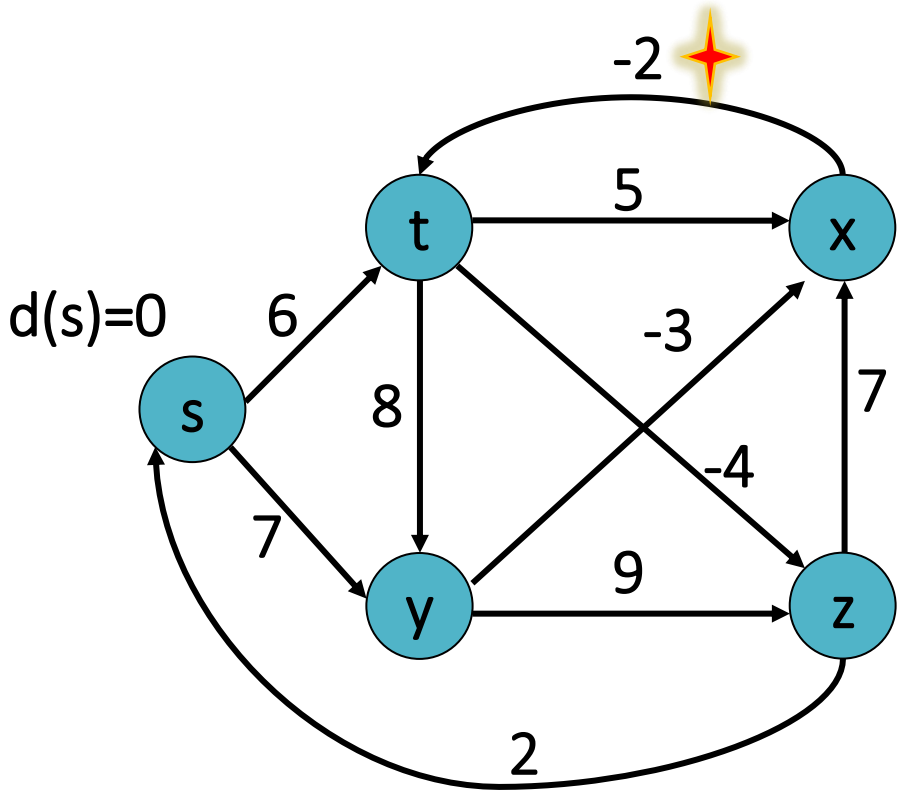


16 > -2 , não muda o rótulo de z

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

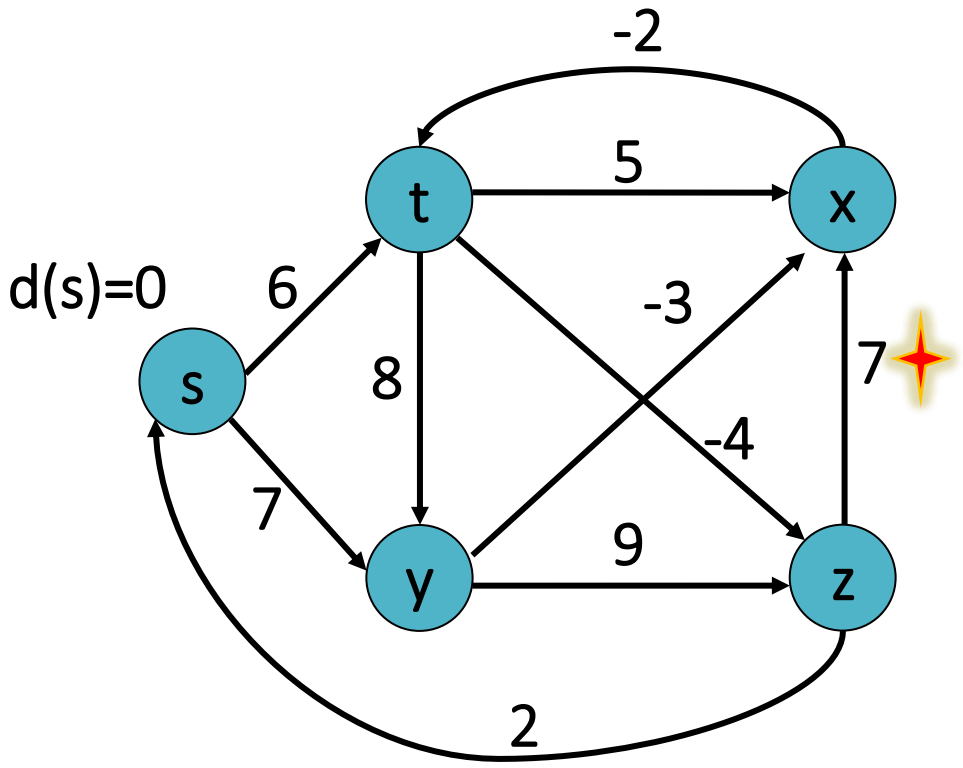


2 = 2 , não muda o rótulo de t

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

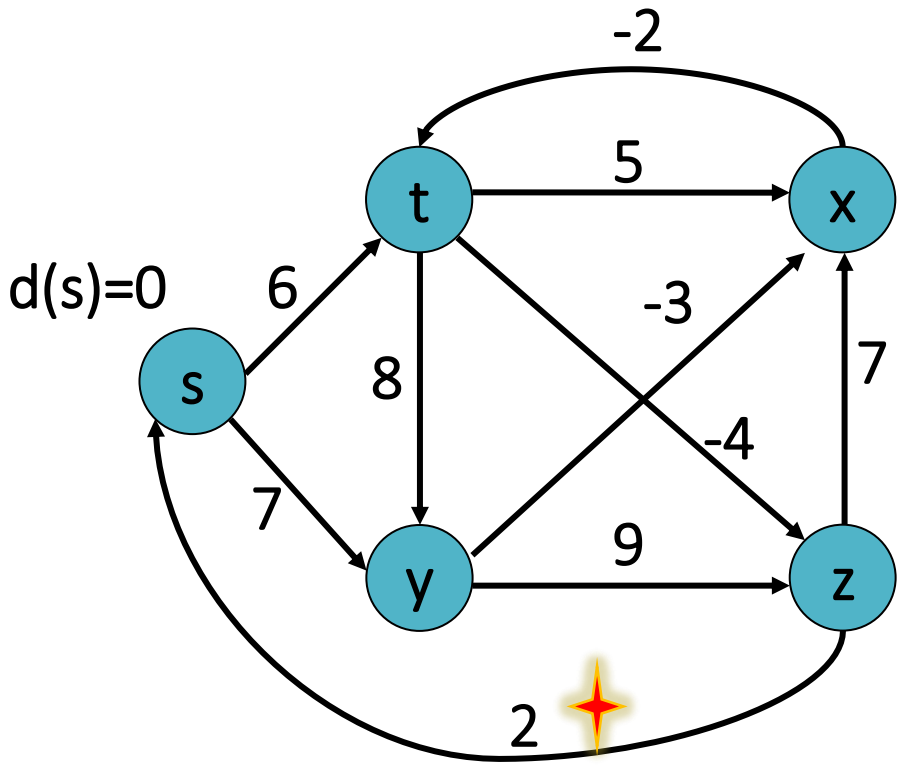


5 > 4, não muda o rótulo de x

	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



0 = 0, não muda o rótulo de s

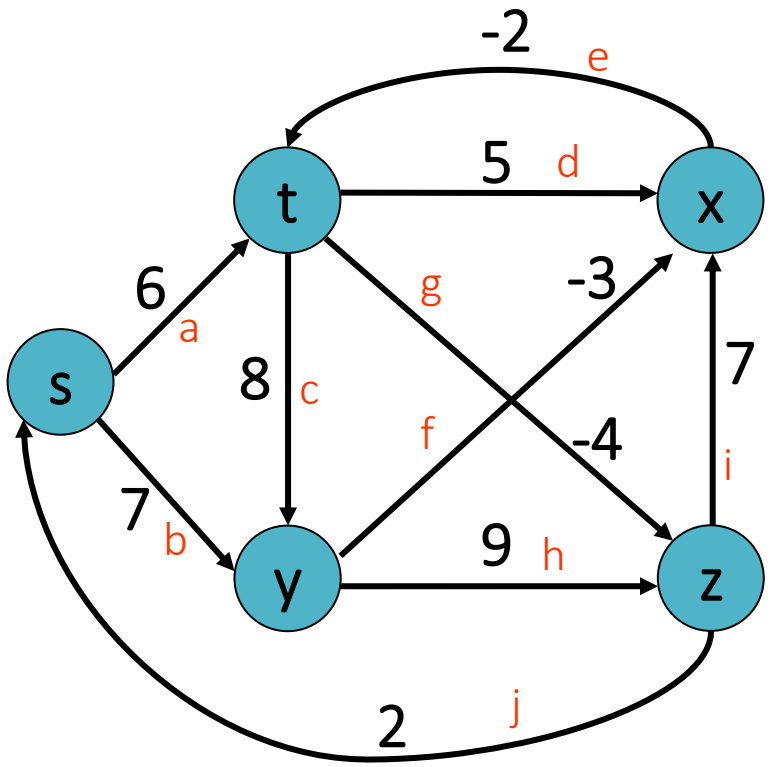
	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

Algoritmo de Bellman-Ford – Exemplo 1

Nova execução do laço não modifica qualquer rótulo

Algoritmo de Bellman-Ford – Exemplo 1

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

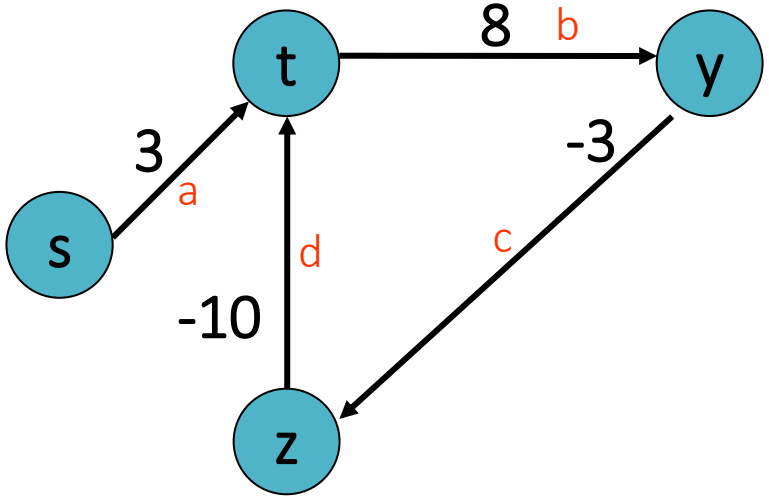


	d(v)	rot(v)
t	2	x
x	4	y
y	7	s
z	-2	t

FIM

Algoritmo de Bellman-Ford – Exemplo 2

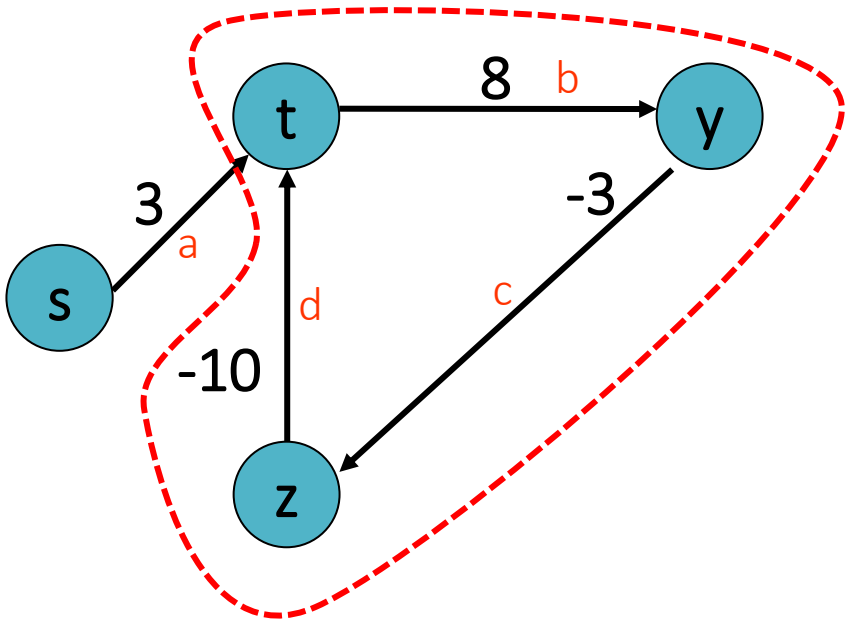
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	∞	NIL
y	∞	NIL
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 2

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

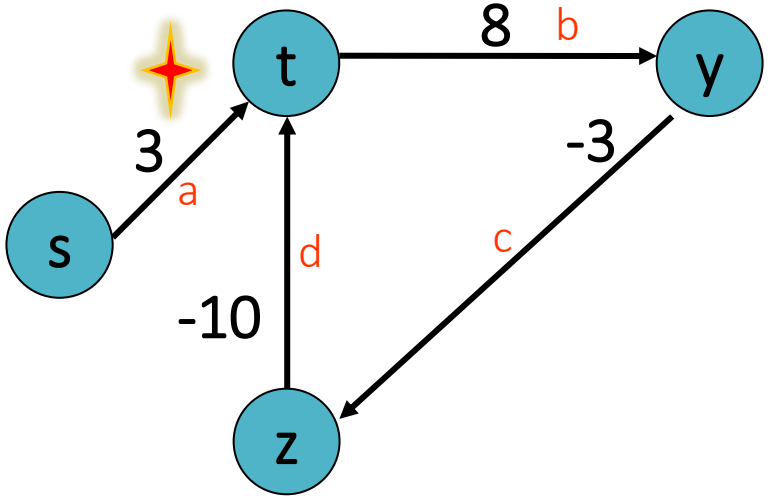


Ciclo de peso negativo

	d(v)	rot(v)
t	∞	NIL
y	∞	NIL
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 2

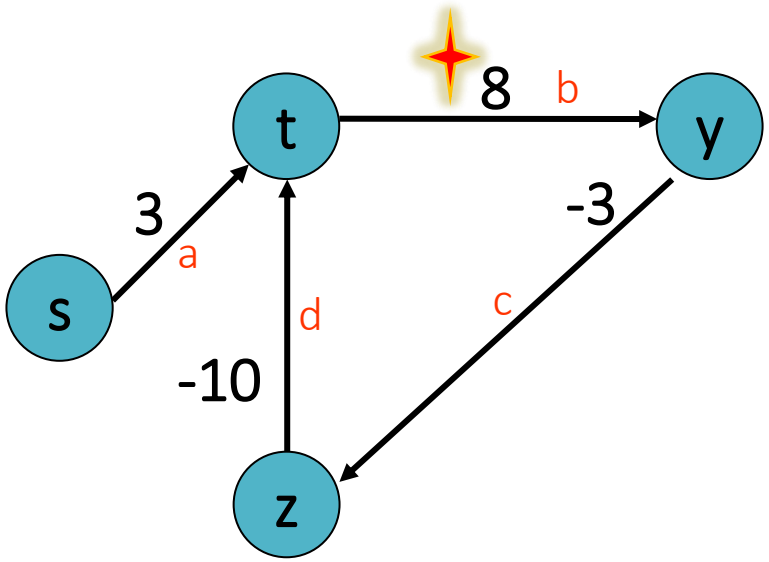
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	3	s
y	∞	NIL
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 2

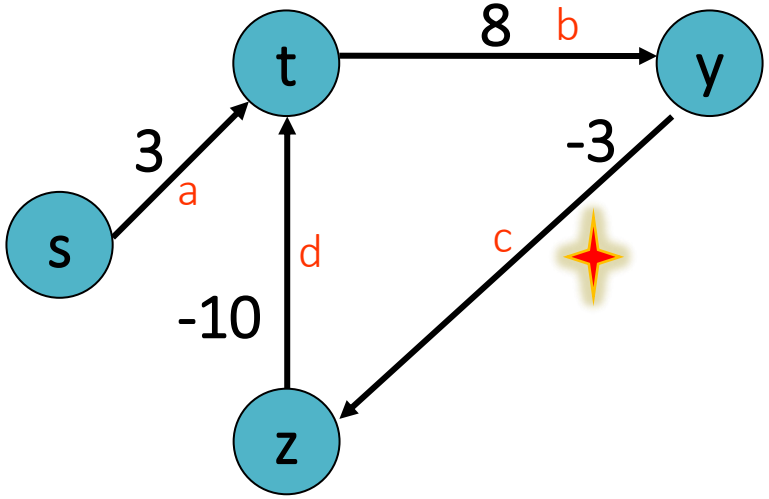
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	3	s
y	11	t
z	∞	NIL

Algoritmo de Bellman-Ford – Exemplo 2

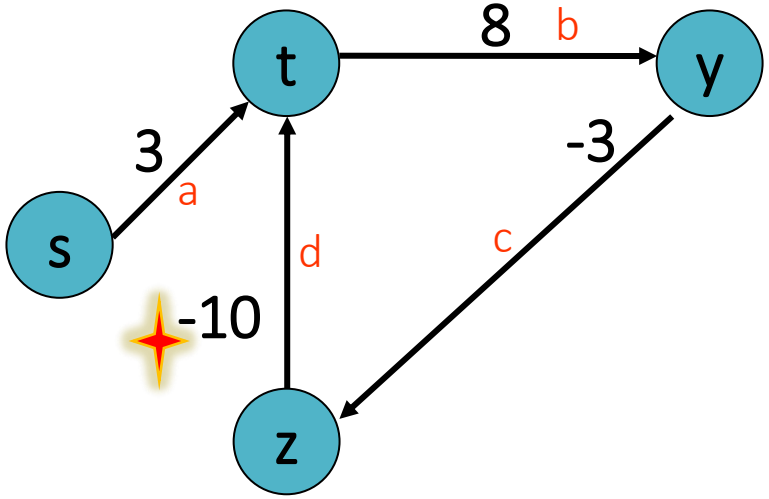
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	3	s
y	11	t
z	8	y

Algoritmo de Bellman-Ford – Exemplo 2

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	-2	z
y	11	t
z	8	y

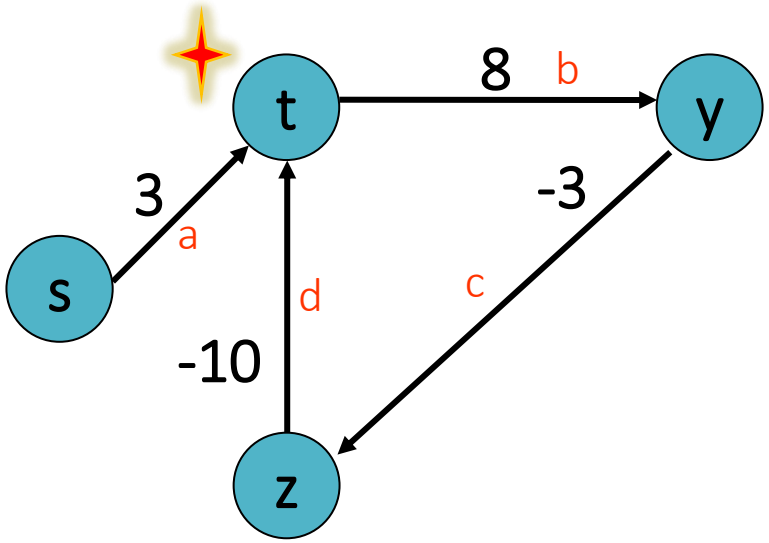
Algoritmo de Bellman-Ford – Exemplo 1

Nova execução do laço

Algoritmo de Bellman-Ford – Exemplo 2

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

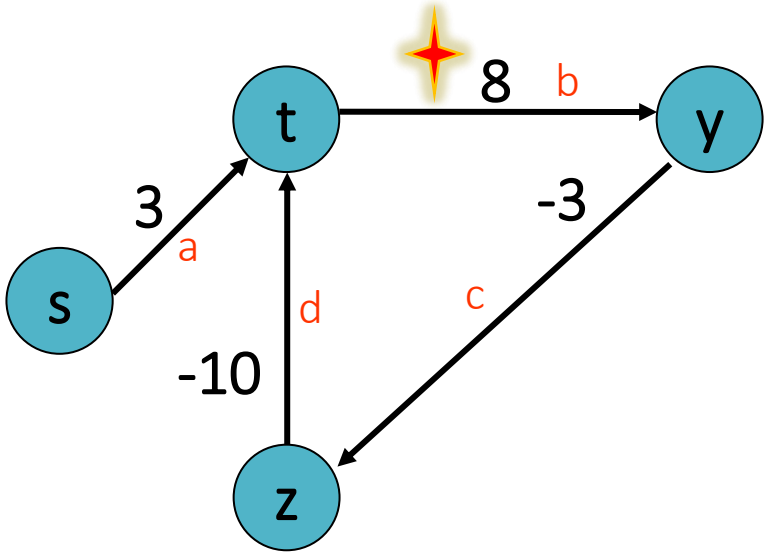
Não modifica o rótulo de t



	d(v)	rot(v)
t	-2	z
y	11	t
z	8	y

Algoritmo de Bellman-Ford – Exemplo 2

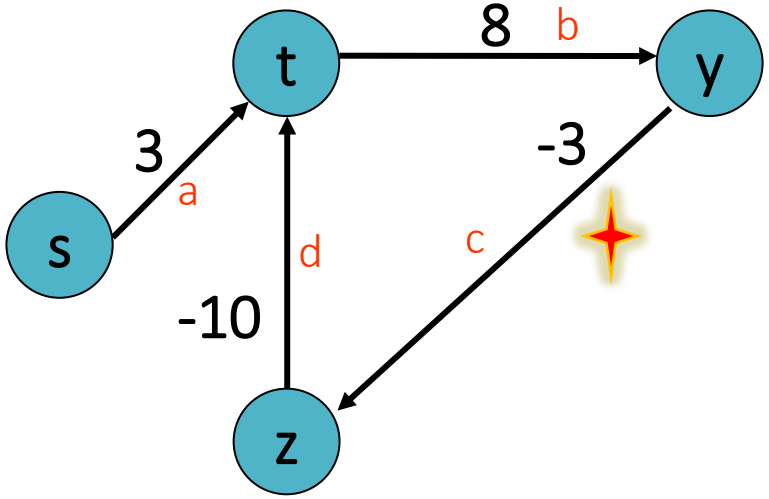
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	-2	z
y	6	t
z	8	y

Algoritmo de Bellman-Ford – Exemplo 2

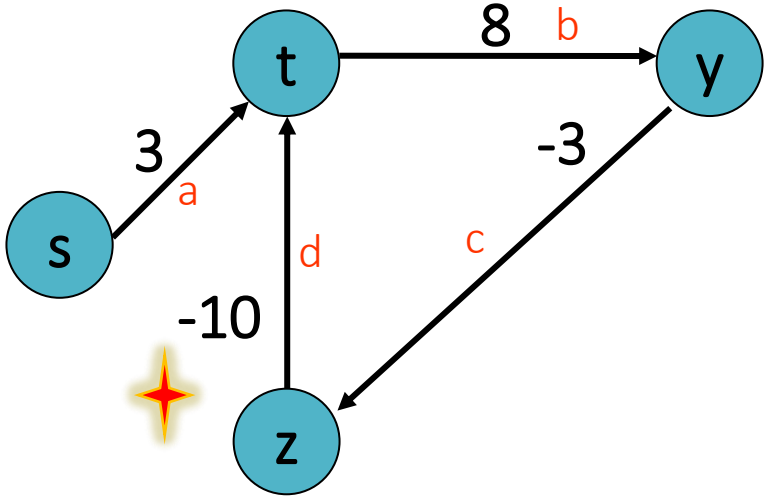
```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	-2	z
y	6	t
z	3	y

Algoritmo de Bellman-Ford – Exemplo 2

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	-7	z
y	6	t
z	3	y

Algoritmo de Bellman-Ford – Exemplo 1

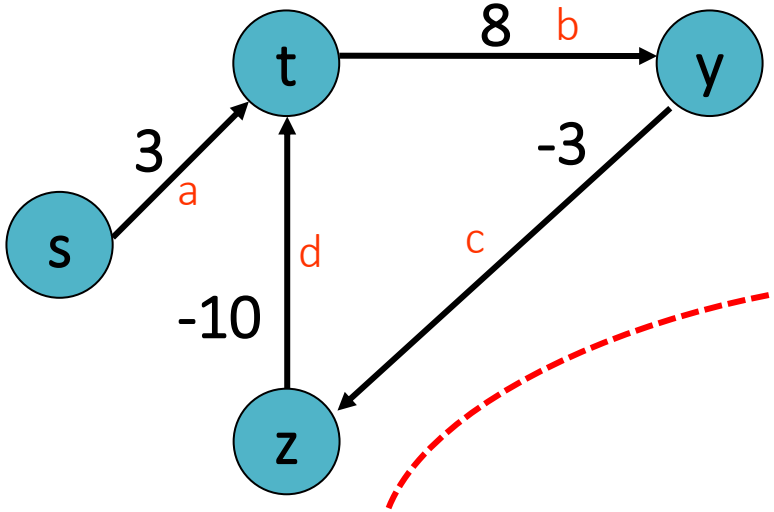
Nova execução do laço vai modificar os rótulos novamente

...

até finalizar as iterações do “para”

Algoritmo de Bellman-Ford – Exemplo 2

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```



	d(v)	rot(v)
t	-7	z
y	6	t
z	3	y

$d(y) > d(t) + w(t,y)$

$6 > -7 + 8$

Ciclo negativo identificado

Algoritmo de Bellman-Ford

Se G não possuir ciclo de peso negativo, então o algoritmo de Bellman-Ford retorna "verdadeiro" para todos os vértices $v \in V$ e encontra o caminho mais curto de uma fonte s a todos os vértices de G. Se G possuir um ciclo de peso negativo o algoritmo retorna "Falso".

Prova de correção do algoritmo: Exercício.

Complexidade

```
Dados G=(V,E), s e w
Para cada v ∈ V faça
    d(v) ← ∞
    rot(v) ← NIL  //predecessor
d(s) ← 0
Para i ← 1 até |V|-1 faça
   para cada aresta (u,v) faça
       se d(v) > d (u) + w(u,v) então
          d(v) ← d (u) + w(u,v)
          rot(v) ← u
Para cada aresta (u,v) faça
   se d(v) > d (u) + w(u,v)
   então retorne(Falso)
Retorne(verdadeiro)
```

Cálculo da complexidade do algoritmo:

Exercício

Algoritmo de Floyd-Warshall

Desenvolvido independentemente por Floyd (1962) e Warshall (1962).

O caminho é desenvolvido de *i* para *j* passando pelo vértice *k*

Técnica – *Programação Dinâmica*

Programação Dinâmica

A programação dinâmica nos fornece uma maneira de projetar algoritmos que:

Sistematicamente exploram todas as possibilidades (corretude);
Armazenam resultados a fim de evitar computação redundante (eficiência);
Definem a solução para o problema em termos da solução de problemas menores.

Programação Dinâmica

É uma técnica para desenvolvermos algoritmos mais eficientes, armazenando resultados parciais.

Essa técnica evita a computação dos mesmos subproblemas repetidamente pois:

Começa a resolver o problema pelos subproblemas de menor tamanho (estratégia *bottom-up*);

A medida que são computados, armazena as respostas;

Quando o mesmo subproblema precisa ser resolvido novamente, apenas consulta o valor armazenado.

Programação Dinâmica

Aplicável a problemas que possuam as propriedades:

- Subestrutura Ótima

O problema pode ser dividido sucessivamente, e a combinação das soluções ótimas dos subproblemas corresponde à solução ótima do problema original.

- Superposição de Subproblemas

O espaço de subproblemas é pequeno, e eles se repetem durante a solução do problema original.

Programação Dinâmica

Vantagens:

Economiza computação em problemas que possuem superposição de subproblemas

Ganho em desempenho.

Desvantagens:

O número de soluções armazenadas na tabela pode crescer rapidamente caso o espaço de soluções não seja pequeno

Alocação de muita memória.

Programação Dinâmica

Etapas para problemas de Otimização

1. Caracterizar a estrutura de uma solução ótima.
2. Definir recursivamente o valor de uma solução ótima.
3. Calcular o valor da solução ótima em um processo “de baixo para cima” (*bottom-up*).
4. Construir a solução ótima a partir das informações obtidas.

Algoritmo de Floyd-Warshall

Considere $l_{ij(m)}$ o peso mínimo de um caminho entre *i* e *j* usando no máximo *m* arestas.

Quando *m* = 0, existe caminho mais curto entre *i* e *j* se e somente se *i* = *j*.

Portanto, $l_{ij(m)} = \begin{cases} 0 \; se \; i = j \\ \infty \; se \; i \neq j \end{cases}$

Para *m* > 1 $l_{ij(m)} = min\left(l_{ij(m-1)}, min\left\{l_{ij(m-1)} + w_{kj}\right\}\right)$

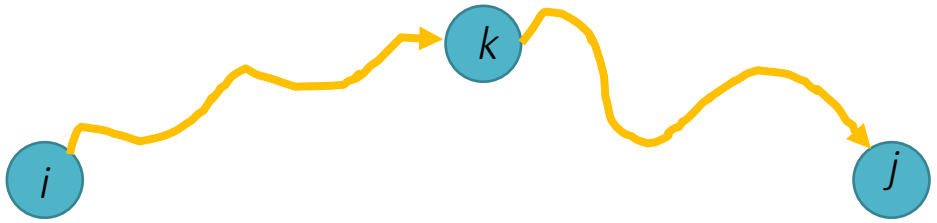
Algoritmo de Floyd-Warshall

Ao invés de considerar arestas, o algoritmo considera os vértices intermediários de um caminho mais curto.

Dado um vértice *k* e um caminho mais curto entre *i* e *j*, podem ocorrer duas situações:

- *k* não pertence ao caminho mais curto entre *i* e *j*. O caminho mais curto que existia antes da consideração de *k* também é um caminho mais curto após a consideração de *k*.-

- *k* pertence ao caminho mais curto entre *i* e *j*. Então o caminho mais curto pode ser desmembrado em dois caminhos mais curtos: *i* até *k* e *k* até *j*.



Algoritmo de Floyd-Warshall

$$Ca\min ho_mais_curto\,(\,G(N,A,D)\,)$$

$$L = D \;\; \{\; L = [l_{ij}]\,\}$$

$$Para\; k = 1\; até\; n \;\; fazer$$

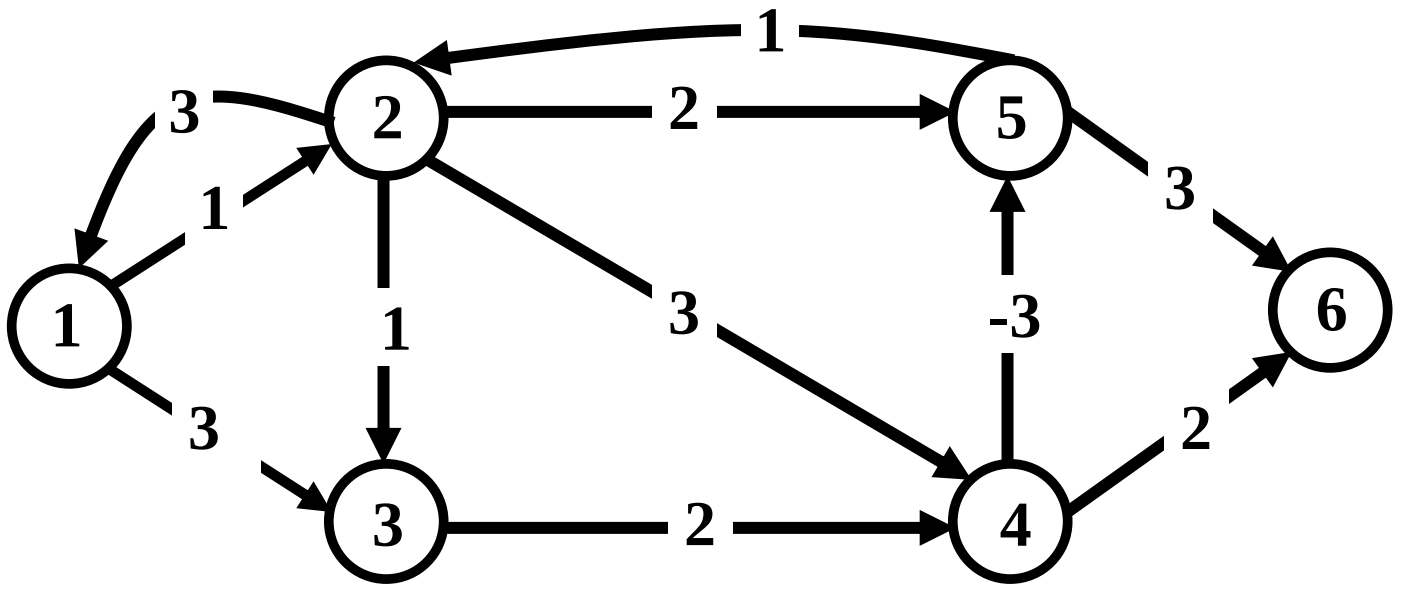
$$Para\; i = 1\; até\; n \;\; fazer$$

$$Para \;\; j = 1\; até\; n \;\; fazer$$

$$l_{ij} = \min(l_{ij}; l_{ik} + l_{kj}) \qquad \Theta(n^3)$$

Algoritmo de Floyd-Warshall

	1	2	3	4	5	6
1	∞	1	3	∞	∞	∞
2	3	∞	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞



Algoritmo de Floyd-Warshall

$k = 1$ – passando pelo vértice 1

$$l_{ij} = \min\left\{ l_{ij}; (l_{i1} + l_{1j}) \right\}$$

$$k = 1;\ \ i = 1;\ \ \ j = 1...6$$

$$l_{11} = min\left\{ l_{11}; (l_{11} + l_{11}) \right\} = \infty$$

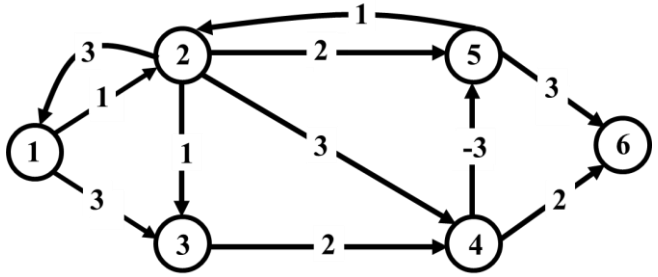
$$l_{12} = min\left\{ l_{12}; (l_{11} + l_{12}) \right\} = l_{12} = 1$$

$$l_{13} = min\left\{ l_{13}; (l_{11} + l_{13}) \right\} = l_{13} = 3$$

.....

$$l_{16} = min\left\{ l_{16}; (l_{11} + l_{16}) \right\} = \infty$$

	1	2	3	4	5	6
1	∞	1	3	∞	∞	∞
2	3	∞	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞



Algoritmo de Floyd-Warshall

$k = 1;\ \ i = 2;\ \ \ j = 1...6$ (2 para j, passando por 1)

$$l_{21} = \min\{ l_{21}; (l_{21} + l_{11}) \} = 3$$

$$l_{22} = \min\{ l_{22}; (l_{21} + l_{12}) \} = 4$$

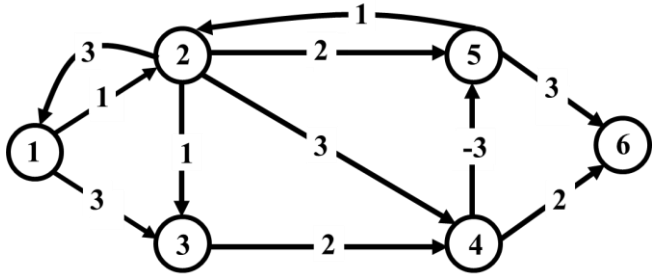
$$l_{23} = \min\{ l_{23}; (l_{21} + l_{13}) \} = l_{23} = 1$$

$$l_{24} = \min\{ l_{24}; (l_{21} + l_{14}) \} = l_{24} = 3$$

$$l_{25} = \min\{ l_{25}; (l_{21} + l_{15}) \} = l_{25} = 2$$

$$l_{26} = \min\{ l_{26}; (l_{21} + l_{16}) \} = \infty$$

	1	2	3	4	5	6
1	∞	1	3	∞	∞	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞

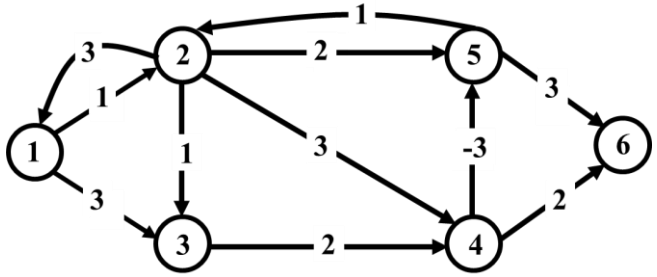


Algoritmo de Floyd-Warshall

k=1; i=3, j=1,...,6
k=1; i=4, j=1,...,6
k=1; i=5, j=1,...,6
k=1; i=6, j=1,...,6

Final para k = 1

	1	2	3	4	5	6
1	∞	1	3	∞	∞	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞



Algoritmo de Floyd-Warshall

$k = 2;\ \ i = 1;\ \ j = 1...6$

$$l_{11} = \min\{ l_{11}; (l_{12} + l_{21}) \} = \mathbf{4}$$

$$l_{12} = \min\{ l_{12}; (l_{12} + l_{22}) \} = l_{12} = 1$$

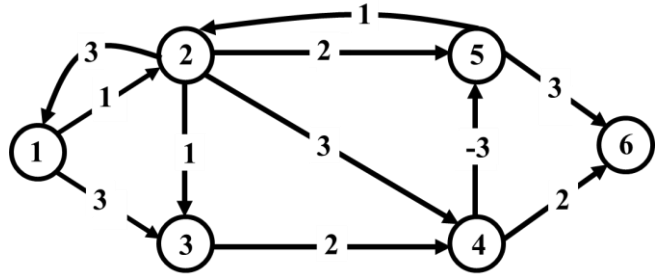
$$l_{13} = \min\{ l_{13}; (l_{12} + l_{23}) \} = (l_{12} + l_{23}) = \mathbf{2}$$

$$l_{14} = \min\{ l_{14}; (l_{12} + l_{24}) \} = (l_{12} + l_{24}) = 4$$

$$l_{15} = \min\{ l_{15}; (l_{12} + l_{25}) \} = (l_{12} + l_{25}) = 3$$

$$l_{16} = \min\{ l_{16}; (l_{12} + l_{26}) \} = \infty$$

	1	2	3	4	5	6
1	4	1	2	4	3	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞



Algoritmo de Floyd-Warshall

$$k = 2;\ \ i = 2;\ \ \ j = 1...6$$

$$l_{21} = \min\left\{ l_{21}; (l_{22} + l_{21}) \right\} = 1$$

$$l_{22} = \min\left\{ l_{22}; (l_{22} + l_{22}) \right\} = l_{22} = 2$$

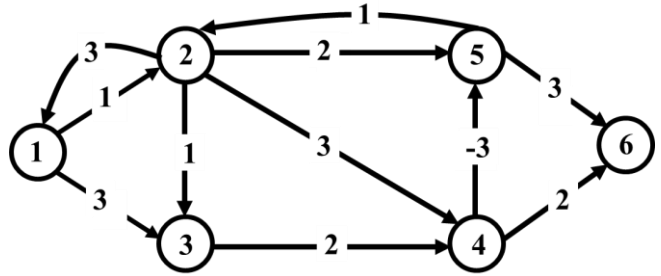
$$l_{23} = \min\left\{ l_{23}; (l_{22} + l_{23}) \right\} = l_{23} = 1$$

$$l_{24} = \min\left\{ l_{24}; (l_{22} + l_{24}) \right\} = l_{24} = 3$$

$$l_{25} = \min\left\{ l_{25}; (l_{22} + l_{25}) \right\} = l_{25} = 2$$

$$l_{26} = \min\left\{ l_{26}; (l_{22} + l_{26}) \right\} = \infty$$

	1	2	3	4	5	6
1	4	1	2	4	3	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	∞	1	∞	∞	∞	3
6	∞	∞	∞	∞	∞	∞

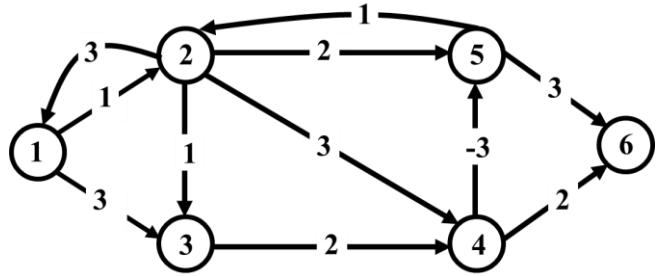


Algoritmo de Floyd-Warshall

k=2; *i*=3, *j*=1,...,6
k=2; *i*=4, *j*=1,...,6
k=2; *i*=5, *j*=1,...,6 ←
k=2; *i*=6, *j*=1,...,6

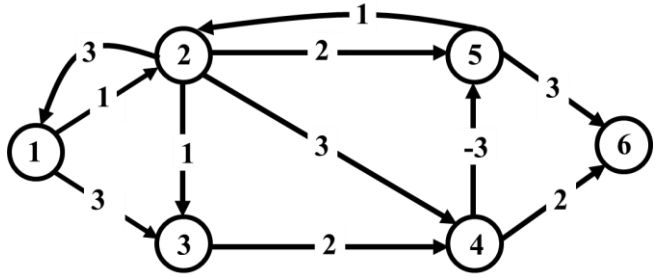
Final para *k* = 2

	1	2	3	4	5	6
1	4	1	2	4	3	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	4	1	2	4	3	3
6	∞	∞	∞	∞	∞	∞



Algoritmo de Floyd-Warshall

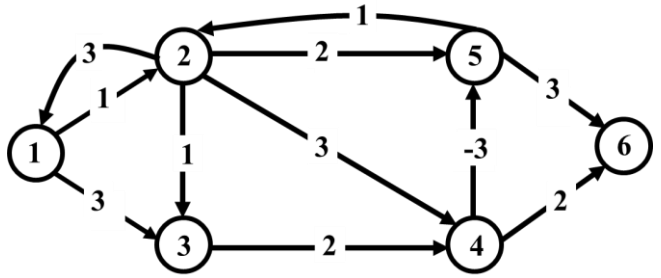
	1	2	3	4	5	6
1	4	1	2	4	3	∞
2	3	4	1	3	2	∞
3	∞	∞	∞	2	∞	∞
4	∞	∞	∞	∞	-3	2
5	4	1	2	4	3	3
6	∞	∞	∞	∞	∞	∞



Matriz *L* para *k* = 3

Algoritmo de Floyd-Warshall

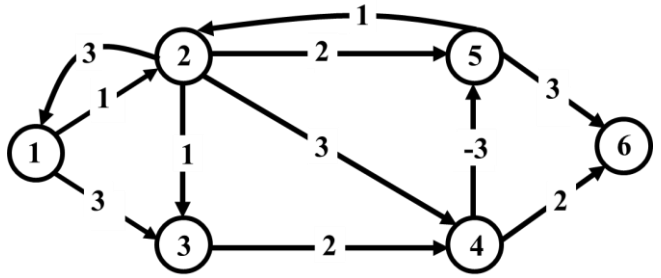
	1	2	3	4	5	6
1	4	1	2	4	3	6
2	3	4	1	3	0	5
3	∞	∞	∞	2	-1	4
4	∞	∞	∞	∞	-3	2
5	4	1	2	4	1	3
6	∞	∞	∞	∞	∞	∞



Matriz *L* para *k* = 4

Algoritmo de Floyd-Warshall

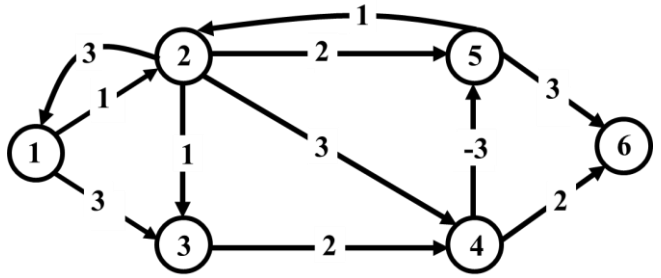
	1	2	3	4	5	6
1	4	1	2	4	3	6
2	3	1	1	3	0	3
3	3	0	1	2	-1	2
4	1	-2	-1	1	-3	0
5	4	1	2	4	1	3
6	∞	∞	∞	∞	∞	∞



Matriz *L* para *k* = 5

Algoritmo de Floyd-Warshall

	1	2	3	4	5	6
1	4	1	2	4	3	6
2	3	1	1	3	0	3
3	3	0	1	2	-1	2
4	1	-2	-1	1	-3	0
5	4	1	2	4	1	3
6	∞	∞	∞	∞	∞	∞

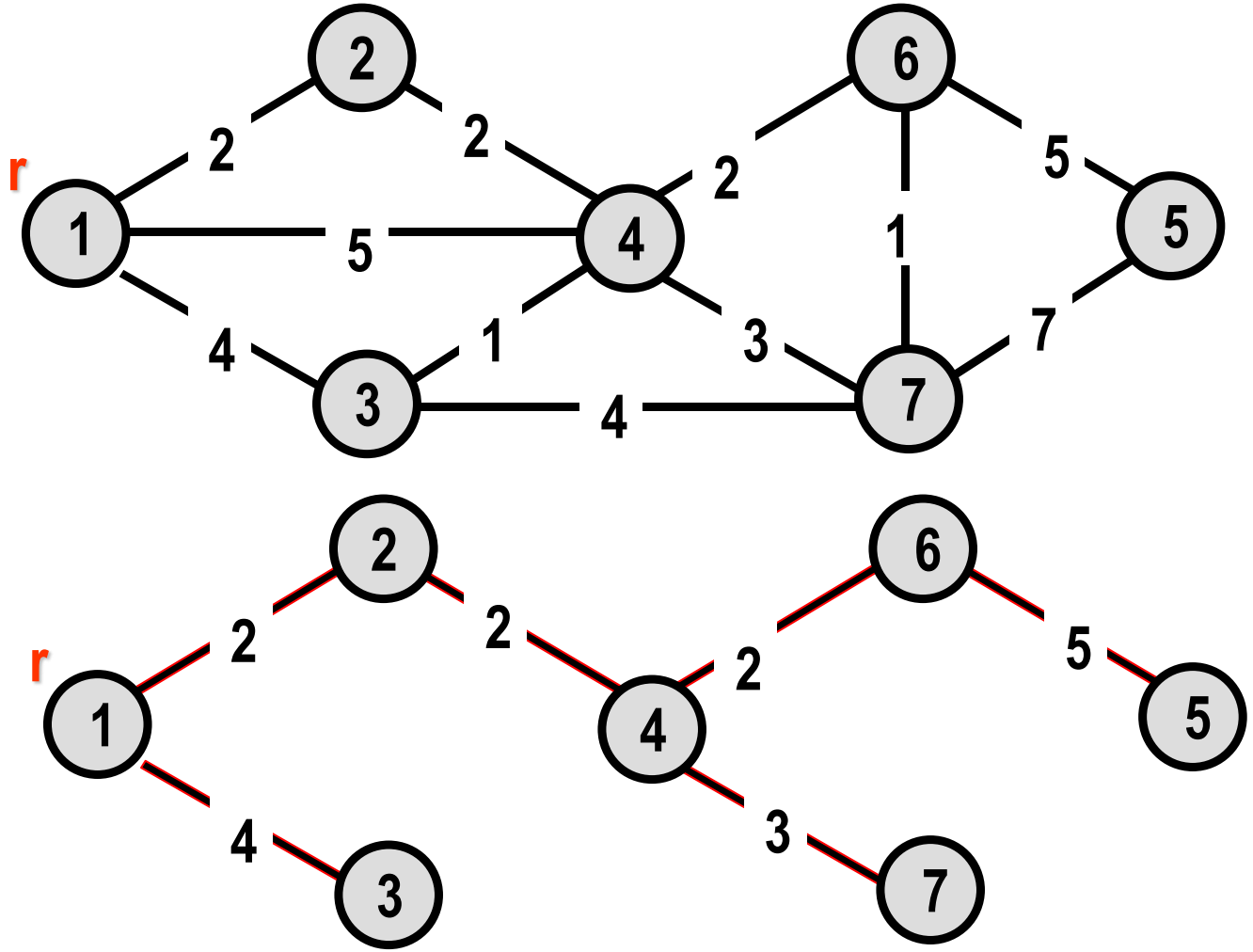


Matriz *L* para *k* = 6

Árvore de Caminhos mais Curtos

Uma árvore de caminhos mais curtos, T_S, é uma árvore com raiz *r* tal que a distância entre *r* e qualquer vértice *u* de T_S é exatamente a mesma que o comprimento do menor caminho entre *r* e *u*.

Árvore de Caminhos mais Curtos



Árvore de Caminhos mais Curtos

Aplicações:

Redes Multicast

VLSI

Instalação de redes de infraestrutura

Referências e outros materiais

Ahuja, R. K., Mehlhorn, K., Orlin, J. B. & Tarjan, R. E. (1990). Faster Algorithms for the Shortest Path Problem. *Journal of the Association for Computing Machinery* **37**(2):213-223.

Bellman, R.E. (1958). On a routing problem, *Quart. Appl. Math.* Vol. 16, pp. 87-90.

Fredman, M. L. & Tarjan, R. E. (1987). Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms. *Journal of the Association Computing Machinery* **34**:596-615.

Ford, L. R. Jr. (1956). Network flow theory, The RAND Corp., P-293, Santa Monica, California.

Hagerup, T. (2000). Improved Shortest Paths in the Word RAM. In: *27th International Colloquium on Automata, Languages and Programming*, 61-72.

Moore, E. F. (1967). The shortest path through a maze, *Proceedings of the International Symposium on the Theory of Switching,* Part II, pp. 285. *The Annals of the Computation Laboratory of Harvard University*, vol. 30, Cambridge.

Referências e outros materiais

Raman, R. (1997). Recent Results on Single Source Shortest Paths Problem. SIGACT News 28:81-87.

Raman. R. (1996). Priority Queues: Small, Monotone and TransDichotomous. In: Proceedings 4th Annual European Symposium Algorithms, Lecture Notes in Computational Science 1136:121-137.

Thorup, M. (1999). Undirected SingleSource Shortest Paths with Positive Integer Weights in Linear Time. *Journal of the Association for Computing Machinery* **46**:362-394.

Thorup, M. (2000a). On RAM Priority Queues. *SIAM Journal on Computing* **30**:86-109.